Hedge Fund
Panel Time Series Forecasting

Course Name: Applied Forecasting Methods

Group Members:

Kaushal Nandaniya (202303036)

Jeetraj Gohil (202303017)

[Member 3 Name] (Roll No: [Roll No])

Instructor Name: Pritam Anand

Institution Name: Dhirubhai Ambani Institute of Information and Communication Technology

Submission Date: May 5, 2026

Abstract

This project addresses a large-scale panel time series forecasting problem derived from anonymized hedge fund data, where the objective is to predict future values of a target variable across multiple heterogeneous series under a weighted evaluation metric. The problem is challenging due to heavy-tailed distributions, extreme weight concentration, non-stationarity, and varying lengths of historical data across series.

The study adopts a diagnostic-driven approach, beginning with extensive exploratory data analysis to understand missingness patterns, distributional properties, and weight imbalance. Stationarity is assessed using ADF and KPSS tests, followed by transformations such as first differencing to stabilize the series. Classical forecasting models, including smoothing methods, AR, MA, ARMA, and ARIMA, are applied to representative series selected based on length, volatility, and weight characteristics. In parallel, deep learning models such as RNN, GRU, and LSTM are implemented using adaptive chronological splits to address limitations of fixed training windows.

The results demonstrate that no single model universally outperforms others across all series. Simple smoothing methods perform well on stable, low-variance series, while differenced ARIMA models and recurrent neural networks show superior performance on volatile and high-weight series. The findings highlight the importance of aligning model choice with data characteristics and evaluation metrics.

Overall, this work emphasizes that effective hedge fund forecasting requires a combination of statistical diagnostics, model adaptability, and careful consideration of panel heterogeneity.

Contents

Section 1. Introduction

1.1 Background and Motivation

Hedge funds rely heavily on quantitative forecasting models to anticipate the behaviour of large numbers of financial instruments simultaneously. The forecasting problem at this scale has three properties that make it a useful subject for an applied forecasting course: panels of thousands of related but heterogeneous series, evaluation metrics that weight rows extremely unevenly because some instruments have far larger economic significance than others, and series of widely varying lengths and statistical character. A model that performs well on average behaviour but degrades on the small set of high-weight rows can be objectively worse than a simpler baseline under the official metric, even if its mean error is lower.

Three properties make the dataset used in this project a particularly clean test bed for methodological work:

- *Anonymisation forces statistical reasoning.* Without semantic feature names we cannot fall back on domain priors; every claim must be supported by a measurable property of the data.
- *The weighting scheme rewards precision on rare rows.* A model that improves average behaviour while degrading on the high-weight rows can lose to a simpler model. This is a mechanically different objective from standard regression and warrants explicit study.
- *The four horizons are coupled.* As we show in Section 4.14, the targets at $H \in \{3, 10, 25\}$ are well approximated by rolling cumulative sums of the $H = 1$ process. This structure is not advertised in the dataset description and admits a modelling strategy that the midway report did not consider.

1.2 Problem Statement

We work with a publicly released Kaggle competition dataset titled *Hedge fund – time-series forecasting*. The competition sponsor releases a panel of anonymised numerical series with engineered features and asks competitors to forecast a continuous target at four lookahead horizons. Identifiers, feature names, and the underlying financial domain are stripped. What remains is a mathematically well-defined supervised forecasting problem with a non-trivial weighting scheme.

Each row of the panel is identified by the tuple

$$(\texttt{code}, \texttt{sub\_code}, \texttt{sub\_category}, \texttt{horizon}, \texttt{ts\_index})$$

and carries a continuous target y_{target}, a non-negative competition weight, and 86 anonymised numerical covariates of the form $\texttt{feature\_a}, \texttt{feature\_b}, \ldots$ The integer time index `ts_index` runs over the contiguous range 1–3601 in the public training partition and 3602–4376 in the held-out test partition. The training panel contains 5,337,414 rows and the test panel contains 1,447,107 rows. The four forecast horizons are $H \in \{1,\ 3,\ 10,\ 25\}$, and there are 36,923 distinct series in train under the 4-tuple key.

The dataset combines three well-known difficulties that make naive baselines a bad guide:

1. **Heavy-tailed target.** The target distribution is centred near zero with an interquartile range of approximately ± 0.05, but the full range runs from roughly -2200 to $+2300$. The 1% and 99% percentiles are at approximately ± 82. Plain mean-squared losses are extremely sensitive to a small number of tail rows.

2. **Concentrated evaluation weight.** The competition metric is sample-weighted, and the weights are extraordinarily skewed: the median weight is approximately 1.7×10^3 while the maximum is 1.4×10^{13}. The top one percent of rows by weight carry approximately 64% of all evaluation mass, and the top five percent carry approximately 92%. Any unweighted analysis can recommend the wrong model.

3. **Limited per-series history at long horizons.** Although the panel is large in aggregate, only about 5% of individual series cleanly cross the validation cutoff at `ts_index` = 2880. Some series have as few as seven training points before the cutoff, which makes per-series classical fitting infeasible for many model families.

1.3 Objectives

This project pursues four explicit objectives:

1. **Establish diagnostic baselines.** Run the full classical diagnostic chain (stationarity, differencing, autocorrelation, decomposition) and report what it implies for modelling choices.

2. **Test classical and deep families on a fair, statistically calibrated protocol.** Replace hand-picked grids with information-criterion-based order selection, replace point estimates with bootstrap confidence intervals, and use chronologically honest splits.

3. **Build a global, panel-aware gradient-boosted model that uses the 86 features and respects the per-horizon target scale.** Show that the H3 weight-dominance finding from the EDA is empirically actionable.

4. **Exploit the multi-horizon coupling.** Test whether fitting only the H=1 process and aggregating to H3, H10, H25 by rolling sum can recover or surpass per-horizon-independent fits.

1.4 Scope of this report

The remainder of this report covers the diagnostic phase, the methodology, and the empirical results. Section 2 formalises the data, panel keys, and chronological constraint. Section 3 develops the official competition metric. Section 4 carries out the full exploratory analysis, including three originality findings about the multi-horizon structure of the panel. Section 5–Section 7 cover classical baselines, deep sequence models, and the global gradient-boosted approach. The probabilistic forecasting work, multi-horizon aggregation experiment, ensembles, comparative analysis, conversion into actions, and future work are added in later chapters as those notebooks land.

Section 2. Data Source and Characteristics

This section fixes the data source, panel keys, and chronological constraints used throughout the project. Data pre-processing details (missing-value handling, normalisation, feature engineering) live in Section 4, which also contains the exploratory analysis the prof's outline requires under that heading.

2.1 Source

The dataset is the publicly released Kaggle competition *Hedge fund – time-series forecasting*, distributed as two parquet files (`train.parquet`, `test.parquet`) totalling approximately one gigabyte. The training partition spans `ts_index` 1 to 3601, the held-out test partition continues from `ts_index` 3602 to 4376. All identifiers, feature names, and underlying financial semantics are anonymised by the competition organiser.

2.2 Panel keys and unit of analysis

A row of the panel has primary keys (`code`, `sub_code`, `sub_category`, `horizon`, `ts_index`). We treat the 4-tuple

$$s \equiv (\texttt{code}, \texttt{sub\_code}, \texttt{sub\_category}, \texttt{horizon})$$

as the natural *series identifier*, and treat `ts_index` as the time axis. There are 23 codes, 180 distinct sub-codes, 5 sub-categories, and 4 horizons in train, yielding 36,923 distinct series. Test uses a narrower sub-code support of 47 distinct values, which is a coverage shift rather than a schema mismatch (every test category level already exists in train).

2.3 The four horizons are coupled

A naive reading of the schema would treat the four horizons of the same (`code`, `sub_code`, `sub_category`) as four independent series, simply distinguished by an extra integer label. Two empirical facts in Section 4 contradict this view:

1. For approximately 1.21 million distinct (`code`, `sub_code`, `sub_category`, `ts_index`) tuples, all four horizons are simultaneously present (Section 4.13).
2. For the same triple, the target at horizon k is closely approximated by the rolling sum of the $H = 1$ target over the next k steps:

$$y_t^{H_k} \approx \sum_{i=0}^{k-1} y_{t+i}^{H_1}.$$

   We verify this on a stratified sample of 500 triples in Section 4.14, and find median Pearson correlations of 0.96, 0.94 and 0.91 at $k = 3, 10, 25$ respectively.

The practical consequence is that fitting four independent regressors per triple discards a strong constraint that the data construction itself imposes. We exploit this constraint in the H1-aggregation experiment scheduled for a later notebook.

2.4 Chronological constraint

The training partition runs over `ts_index` $\in [1, 3601]$ and the test partition over $[3602, 4376]$. Because test is a strict temporal suffix of train, any internal validation strategy must also be chronological, with all training points strictly preceding the validation points in time. Random or leave-one-series-out cross-validation would mix future information into earlier states and overstate generalisation.

We use a single canonical chronological partition throughout the project, fixed in the shared utilities at `cf.splits`:

- **Train:** `ts_index` ≤ 2880.
- **Meta** (used only for fitting ensemble weights): $2880 <$ `ts_index` ≤ 3240.
- **Holdout** (final scored window): $3240 <$ `ts_index` ≤ 3601.
- **Test** (`ts_index` ≥ 3602): unlabeled, used only for submission-style sanity checks.

The cutoff `ts_index` $= 2880$ matches the cutoff used in the midway report, preserving comparability with prior diagnostic figures. The meta/holdout boundary at 3240 places approximately one third of the post-cutoff window into the meta slice, leaving the remaining two thirds for final scoring.

2.5 Eligibility for per-series studies

Some classical model families (ARIMA, ARMA, MA, AR with order ≥ 2) require a minimum training history. For per-series diagnostic studies we restrict attention to series that both cross the train/validation cutoff and have at least 120 training rows. There are 1,930 such *eligible* series in the panel, roughly 5% of the 36,923 total. Global pooled models and the H1-aggregation experiment use every row whose `ts_index` falls inside the relevant split, so they are not constrained by per-series length.

Section 3. The Weighted Skill Score

The competition is decided by a single scalar: the *weighted skill score*. Because the rest of the project uses this metric for every model comparison, ensemble weight, and bootstrap interval, it deserves a dedicated section. We state the formula, give the intuitive reading, and note the two practical consequences that carry through every modelling decision.

3.1 Definition

Given truths y_i, predictions $\hat{y}_i$, and non-negative weights w_i for $i = 1, \ldots, n$, the weighted skill score is

$$\text{score}(y, \hat{y}; w) \;=\; \sqrt{1 \;-\; \text{clip}_{[0,1]}\left(\frac{\sum_i w_i \, (y_i - \hat{y}_i)^2}{\sum_i w_i \, y_i^2} \right)} \tag{1}$$

where $\text{clip}_{[0,1]}(x) = \min(\max(x, 0), 1)$. The function returns 1 for a perfect prediction and 0 when the prediction is no better than the all-zero forecast.

3.2 Intuitive meaning

Strip away the symbols and the score answers a single question: *how much of the weighted target "energy" is left unexplained by the prediction?*

The fraction inside the square root, $\sum w_i (y_i - \hat{y}_i)^2 \,/\, \sum w_i y_i^2$, is exactly that — the unexplained share of weighted squared truth. It is zero when the prediction is perfect and one when the prediction is identically zero. The clip caps it at one so that no model can score worse than the trivial all-zero forecast, and the outer square root puts the result on a familiar correlation-like scale where 1 is excellent, 0 is no skill, and intermediate values read roughly the way a Pearson correlation of the same magnitude reads.

A few useful reference points:

- **Skill** $= 1$ — perfect predictions on every weighted row.
- **Skill** $= 0$ — the model is no better than predicting zero. This is the natural "did you learn anything" threshold.
- **Skill** ≈ 0.5 — roughly three quarters of the weighted target energy is still unexplained. In feel, this is comparable to a Pearson correlation of 0.5: a real but modest signal.
- **Skill** ≈ 0.95 — only ten percent of energy unexplained. Strong, near the practical ceiling for a competitive entry.

The reason the metric is built around *weighted* energy rather than raw error is that hedge-fund-style panels have wildly different target scales across instruments. Dividing by $\sum w_i y_i^2$ neutralises that scale: a 1% relative error on a low-volatility row and a 1% relative error on a high-volatility row contribute proportionally to the score. The weights w_i themselves let the organiser encode importance — and in our panel those weights are extraordinarily concentrated, so the score is effectively decided by how well the model fits a small set of high-weight rows.

3.3 Practical consequences for this project

Equation (1) is implemented once in the shared utility `cf.metrics.weighted_skill_score` and used by every later notebook for both model selection and final scoring. The function `cf.metrics.bootstrap_skill` returns 95% confidence intervals by row resampling; we report bootstrap intervals alongside every leaderboard number to avoid declaring winners that are within sampling noise of each other.

Two structural observations carry through to every subsequent modelling decision:

- **High-weight rows decide the leaderboard.** Because approximately 64% of weight sits in the top 1% of rows (Section 4.4), the score is essentially a measure of how well the model predicts those few rows. We must therefore train with sample weights, not just evaluate with them.
- **Plain RMSE will lie.** A model that improves average behaviour at the cost of a small degradation on a high-weight row can lose to a simpler model. We never report unweighted RMSE as a primary headline number.

Section 4. Data Pre-processing and Exploratory Analysis

This section reports the exploratory analysis of the panel before any modelling. The aim is twofold. First, to establish the basic statistical shape of the data so that later modelling choices can be motivated rather than guessed. Second, to surface three structural findings about the panel — multi-horizon coupling, the cumulation hypothesis, and per-horizon weight dominance — that the midway report did not document but that materially change the modelling space. All numerical results below are produced by notebook `00_data_structure.ipynb` and the corresponding figures are saved as both PNG and PDF assets.

4.1 Schema, panel scale, and train–test comparability

The training panel contains 5,337,414 rows, organised by 23 distinct `code` values, 180 distinct `sub_code` values, and 5 `sub_category` values across 4 horizons. The integer time index `ts_index` runs over the contiguous range $[1, 3601]$. The test partition continues from `ts_index` $= 3602$ to 4376 with 1,447,107 rows, so test is a strict temporal suffix of train. Treating the 4-tuple (`code`, `sub_code`, `sub_category`, `horizon`) as the natural series identifier yields 36,923 distinct series.

The only columns present in train but not in test are y_{target} and `weight`, as expected. Every category level seen in test exists in train, so there is no cold-start schema issue. The sub-code support narrows from 180 in train to 47 in test — a coverage shift rather than a schema mismatch.

4.2 Missing values

Missingness is computed on a 500,000-row stratified sample to keep memory in check. Of the 86 anonymised features, 48 contain at least one missing value, but the maximum missing rate across all columns is approximately 12.5%. Figure 1 shows the top fifteen offenders.

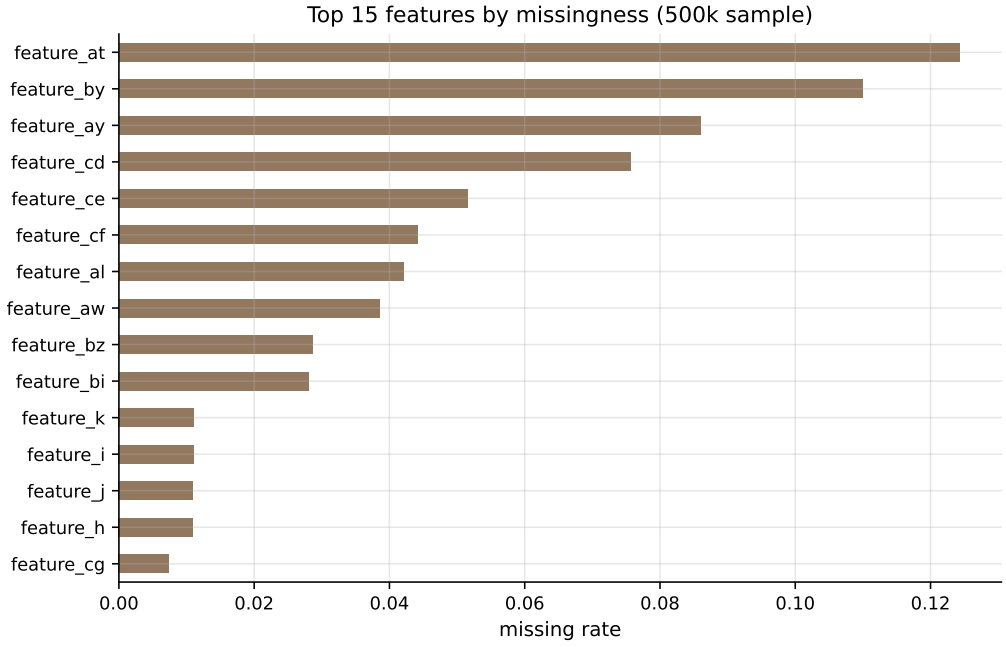

Figure 1: Top fifteen features by missingness, on a 500,000-row stratified sample of train. The maximum missing rate is approximately 12.5%, low enough that median imputation inside a fold is sufficient and there is no need to drop columns or use complex imputers.

Reading. Missing rates are bounded; within-fold median imputation suffices. No separate imputation strategy is developed.

4.3 Target distribution

The target y_{target} is centred very close to zero (median -0.0006, IQR ± 0.05), but has extremely heavy tails. The 1% and 99% quantiles sit at approximately -82.8 and $+62.9$ respectively, while the full range runs from approximately -2202 to $+2314$.

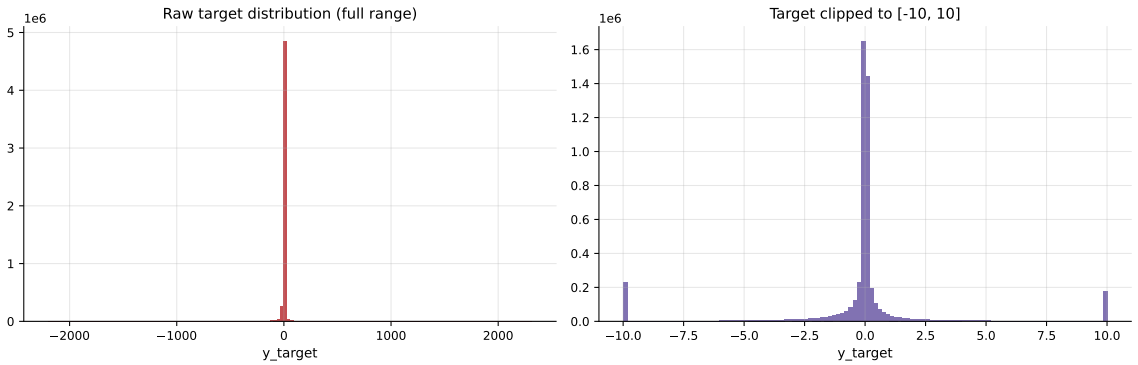

Figure 2: Raw target distribution (left) and the same distribution clipped to $[-10, 10]$ (right). Most of the mass concentrates near zero, with rare extreme tails up to the order of ± 2200.

Reading. The shape disqualifies plain unweighted MSE as a sensible loss. Tails are real outcomes, not corruption, so we cannot simply clip them; combined with the weighting scheme below, this is the central methodological constraint.

4.4 Weight distribution and metric concentration

The competition weights are extraordinarily skewed. The median weight is 1,699, while the maximum is 1.39×10^{13}. Figure 3 shows the weight distribution clipped at the 99th percentile (left) and the cumulative-share Lorenz-style curve (right). The top 1% of rows carry 64.2% of total weight; the top 5% carry 92.6%; the top 10% carry 98.3%.

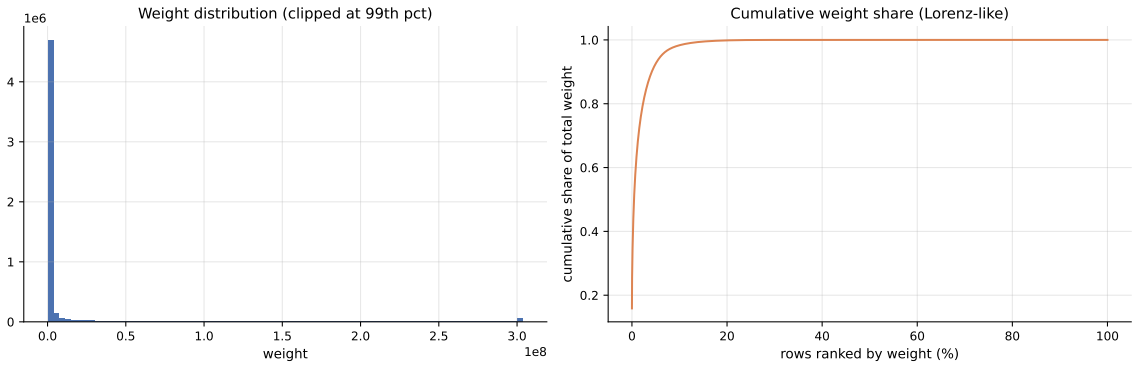

Figure 3: Weight distribution clipped at the 99th percentile (left) and cumulative share of total weight ranked by weight (right). Approximately 64% of total weight is concentrated in the top 1% of rows; 92% in the top 5%.

Reading. Unweighted evaluation describes a different competition. The weighted skill score (Section 3) is our only headline metric, and a model that accurately predicts the top one percent

of rows has more leverage on the leaderboard than a uniformly mediocre one.

4.5 Series coverage and split feasibility

Not every series has enough history to support time-based validation. Figure 4 shows the distribution of per-series lengths. Out of 36,923 distinct series, only 1,930 (approximately 5%) cleanly cross the train/validation cutoff at `ts_index` = 2880. Many series end before the cutoff, contributing zero validation rows.

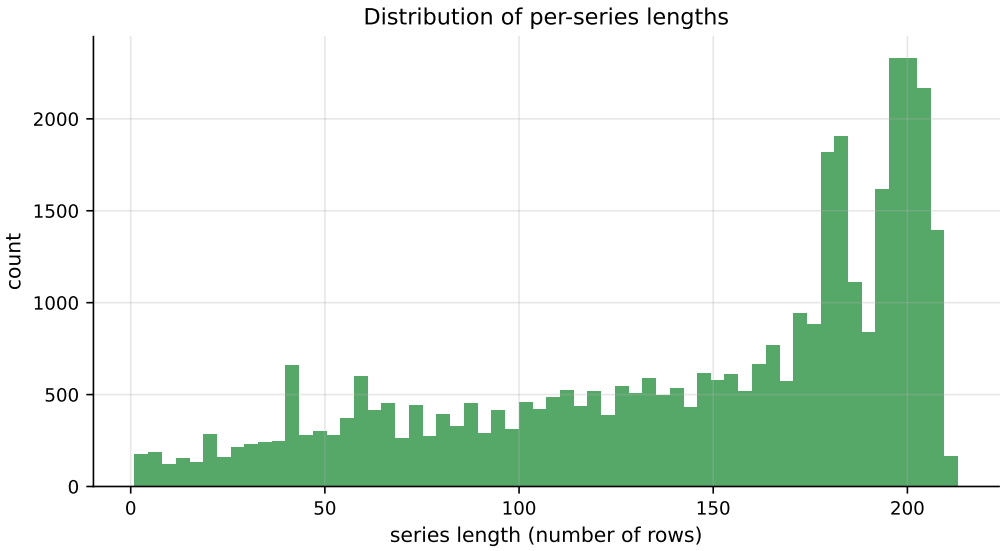

Figure 4: Distribution of per-series lengths. Most series are short, and the tail of long series carries the structural signal that classical fits can exploit.

Reading. Per-series classical fitting is feasible only on the eligible subset (cross the cutoff with at least 120 training rows). Global pooled models are not length-constrained — one reason the comparison between local classical and global ML approaches is not perfectly apples-to-apples.

4.6 Representative-series selection

For every per-series figure in this project we plot the same four series. They are selected with explicit reasons rather than at random, in order to span four distinct stress regimes: long history, large weight, high variability, and near-flat behaviour. Table 1 lists the four chosen series. They match the selection used in the midway report, preserving visual continuity.

Table 1: Four representative series chosen for per-series studies. Each series probes a different regime of the panel.

Reason	Series key (code/sub_code)	Horizon	Length
Longest history	X9BZ68VQ / OYJGNSQK / DPPUO5X2	1	212
Highest total weight	SJZP0OVU / OYJGNSQK / NQ58FVQM	25	156
Most volatile	W4S29LF4 / KL66VIS3 / PHHHVYZI	25	162
Most stable	SJZP0OVU / OYJGNSQK / NQ58FVQM	1	170

Figure 5 shows the raw y_{target} trajectory of each chosen series with the train/validation cutoff marked. The four panels look profoundly different: the most-volatile series swings between -500 and $+500$, while the most-stable series occupies a numerical range narrower than 0.001.

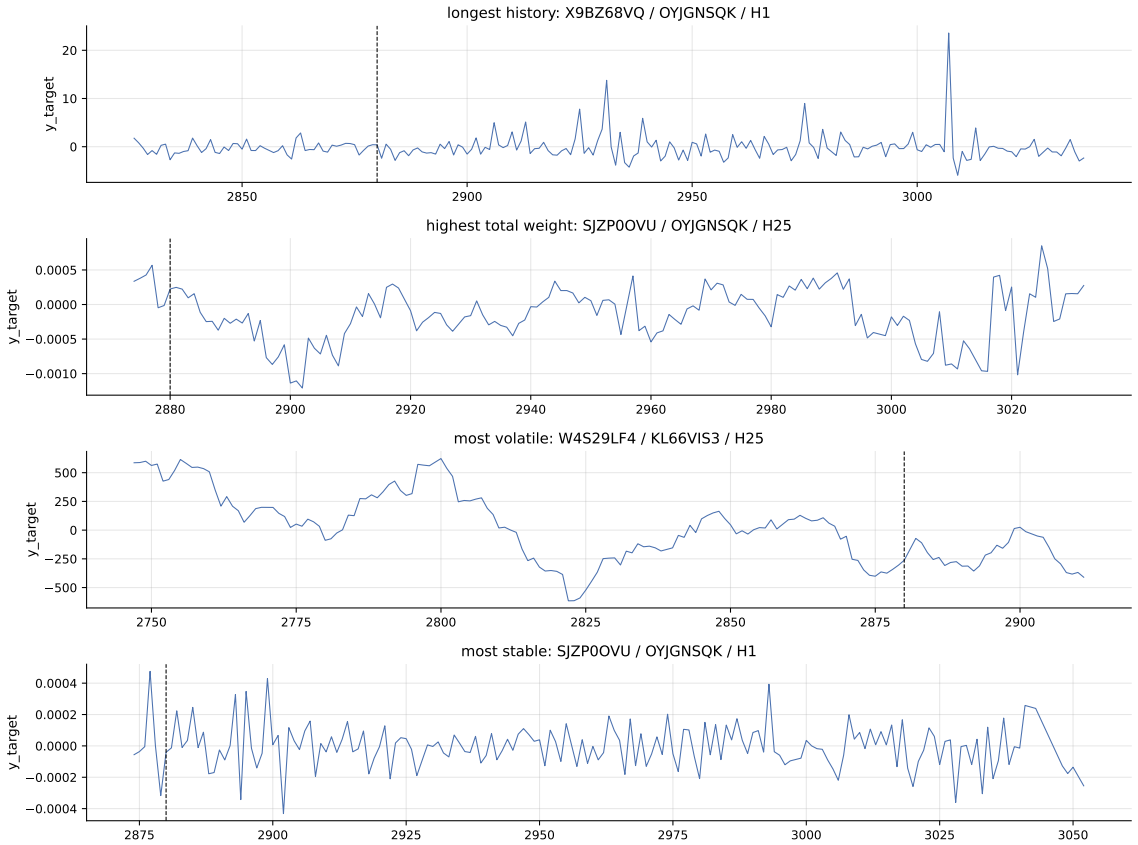

Figure 5: Raw y_{target} for the four chosen representative series. The dashed vertical line marks `ts_index` = 2880, the train/validation cutoff. The four panels span the four stress regimes that any model has to handle.

Reading. A one-size-fits-all model will struggle: the most-stable series is essentially numerical noise, while the most-volatile series has obvious cyclical structure. Forecast performance across these four panels is one of the cleanest diagnostics for whether a model family generalises or only fits one regime.

4.7 Stationarity diagnostics

Classical ARIMA-family forecasting assumes stationarity, or stationarity-after-differencing. We therefore run the Augmented Dickey–Fuller test on a stratified sample of 200 eligible series. The ADF test takes the null hypothesis that the series has a unit root (non-stationary); we reject at the 5% level when the p-value is below 0.05. Figure 6 summarises the results.

Approximately 70.5% of sampled series reject the unit-root null at the 5% level. The picture is more nuanced when broken down by horizon: H1 series are more often stationary than longer-horizon cumulative series, which is consistent with the multi-horizon coupling we will document in Section 4.14.

ADF and KPSS have opposite null hypotheses, so running both gives a more honest joint verdict: ADF takes non-stationarity as the null, while KPSS takes stationarity as the null. We classify a series as *stationary* when ADF rejects and KPSS does not, as *unit root* when ADF fails to reject and KPSS rejects, and as *inconclusive* otherwise. Figure 7 reports the joint distribution.

The stationary share dominates, but the inconclusive bucket is large enough that one-size-fits-all claims about raw levels would be unsafe. We need a repair step (differencing) for the unit-root and inconclusive cases.

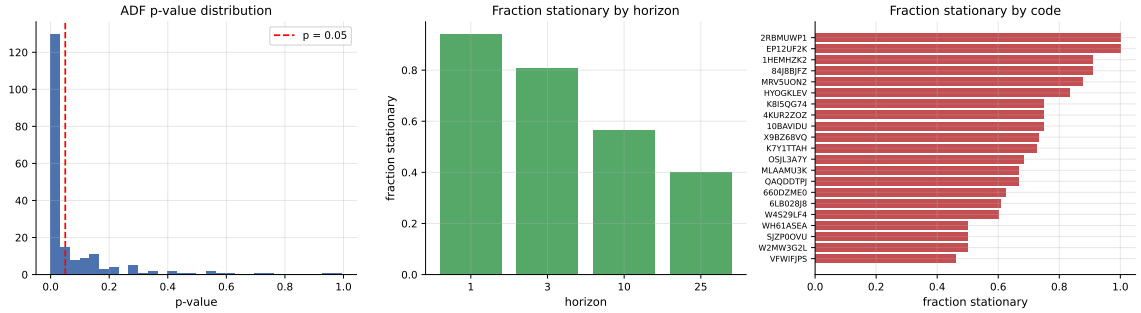

Figure 6: ADF results on 200 sampled eligible series. Left: p-value distribution with the 0.05 threshold marked. Centre: fraction stationary by horizon. Right: fraction stationary by code.

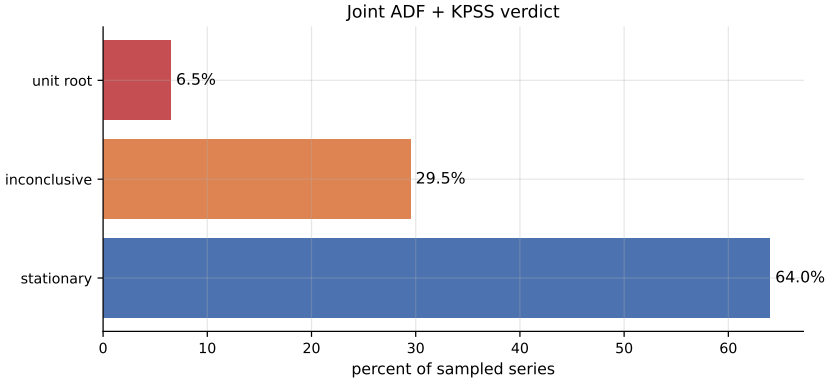

Figure 7: Joint ADF + KPSS verdict on 200 sampled series. Stationary 64.0%, inconclusive 29.5%, unit root 6.5%.

Figure 8 re-runs the ADF test on the first-differenced series. The stationary share rises from 70.5% on raw levels to 98.5% after differencing.

Reading. The diagnostic chain motivates ARIMA with $d = 1$ as the default for the classical study and justifies engineered differenced features for global ML models. The 98.5% post-difference stationarity also bears on the cumulation finding in Section 4.14: differencing a higher-horizon target partially inverts the rolling-sum construction back into the H1 process.

4.8 Visual stationarity — rolling statistics

Statistical tests can disagree on short or noisy series, so a rolling-window view is a useful visual check. Figure 9 shows a 24-step rolling mean and rolling standard deviation on the raw four representative series. The most-volatile series shows clear drift in level and bulging variance; the longest-history series drifts upward; the high-weight and most-stable series occupy near-flat ranges where the rolling mean barely moves.

Figure 10 repeats the same diagnostic on the first-differenced versions of the same series. The rolling means now hug zero and the rolling standard deviations flatten across the cutoff, visually confirming the statistical jump from the previous subsection.

Reading. Differenced series behave like the white-noise regime we want before fitting a differenced classical model.

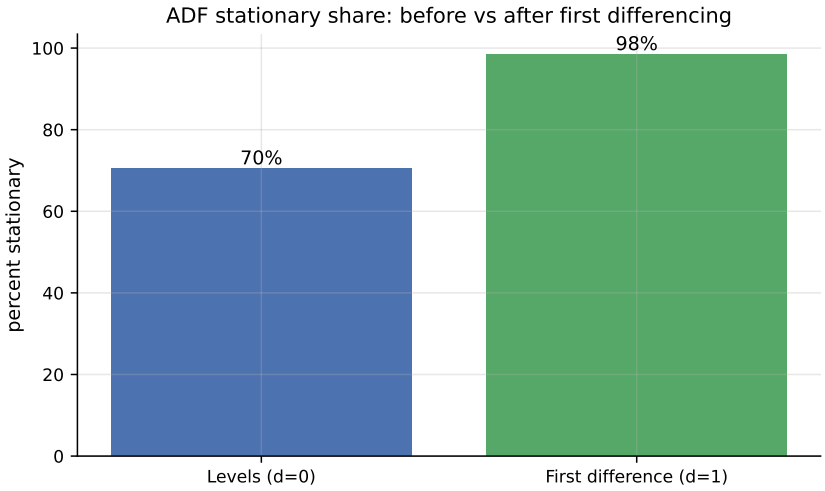

Figure 8: ADF stationary share at the 5% level: levels (left, $d = 0$) versus first differences (right, $d = 1$). Differencing pushes the share to nearly 100%.

4.9 Lag structure — ACF and PACF

The autocorrelation function (ACF) shows correlation between the series and lagged copies of itself; the partial autocorrelation function (PACF) isolates each lag's direct contribution after controlling for shorter lags. A sharp PACF cutoff suggests an autoregressive order; a sharp ACF cutoff suggests a moving-average order. Rather than relying on a single anchor series, we aggregate the absolute ACF and PACF across all 200 sampled eligible series (Figure 11). The panel-level view exposes which lags are reliably informative across the panel rather than artefacts of one trajectory.

Reading. Short-memory dynamics dominate (lags 1–3 carry most of the structure with rapid decay thereafter), and there are no clean seasonal bumps. The lag feature set for the global ML models therefore prioritises lags 1, 2, 3, 5, 10 and skips seasonal indicators.

4.10 STL decomposition

Seasonal-Trend decomposition using LOESS (STL) splits a series into trend, seasonal, and residual components. STL requires a fixed seasonal period as input. The midway report used period = 24 without justification, which is methodologically weak: any STL output is only as meaningful as the period plugged into it. Here we instead select the period from the data using a periodogram of the linearly-detrended anchor series and pick the dominant non-zero frequency. If the spectrum is flat (no concentrated peak) we report that fact directly — a flat spectrum is itself the conclusion that strong seasonality is absent.

Figure 12 shows the periodogram on the longest-history anchor. The spectrum is approximately flat: there is no dominant peak, the strongest non-zero peaks are scattered across short periods (3–8 steps), and the top three frequencies together hold only about 11% of total power. By contrast, a genuinely seasonal series typically concentrates 50%+ of its power into one or two frequencies. The FFT-selected period for STL is 4, but it is selected from a spectrum that does not support a seasonal interpretation in the first place.

Figure 13 shows the resulting STL decomposition with period = 4. The seasonal amplitude is comparable to the trend amplitude, but this is an artefact of using a very short period: at $p = 4$, STL is decomposing the short-memory autocorrelation already surfaced in the panel ACF as if it were seasonality. The structurally informative finding is the flat spectrum, not the STL panels.

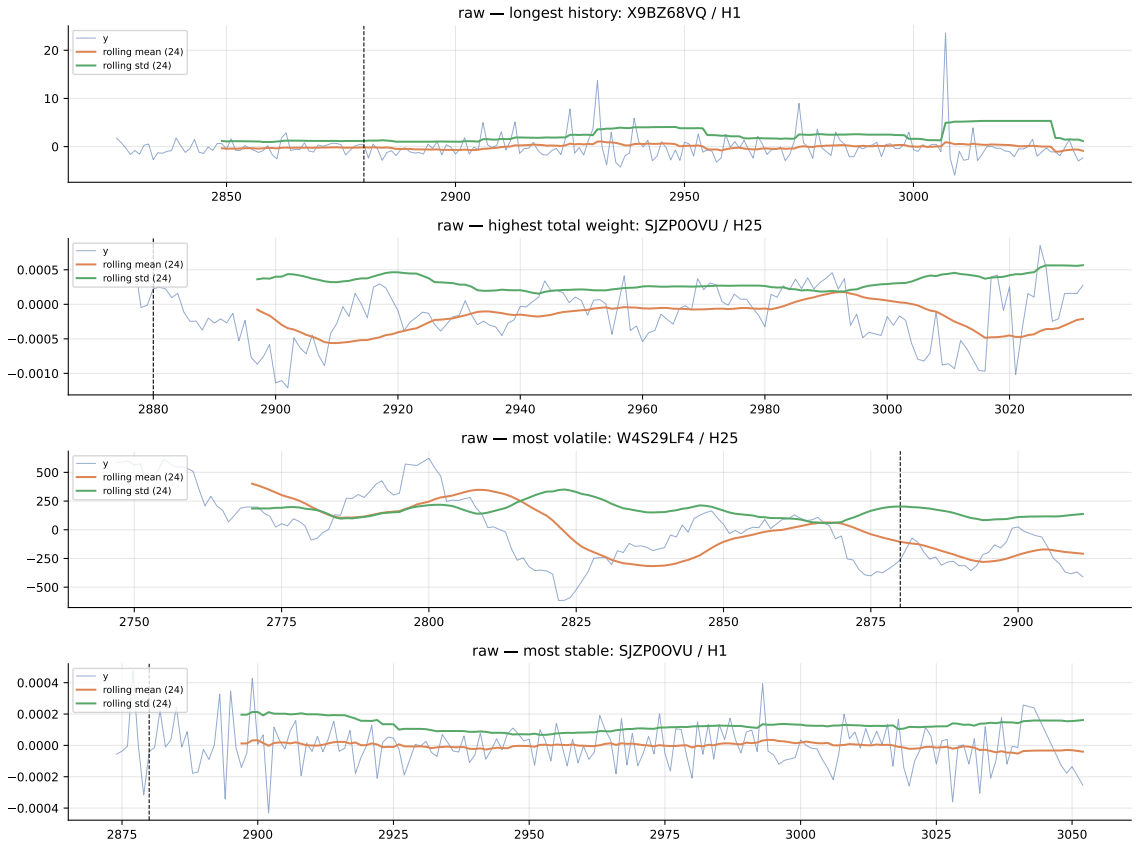

Figure 9: Rolling mean and rolling standard deviation (window = 24) on the raw four representative series. The most-volatile series shows clear drift in level and bulging variance.

Reading. The flat periodogram is the headline. There is no exploitable periodic structure in this panel, so modelling effort should go into short-memory dynamics (lag features 1, 2, 3, 5, 10) rather than seasonal indicators. The STL decomposition is reported because it is part of the standard EDA toolkit, but its seasonal panel is short-memory autocorrelation in disguise, not real seasonality.

4.11 Feature audit

The 86 `feature_*` columns are anonymised and we cannot interpret them by name. We can, however, audit them statistically. On a 200,000-row stratified sample we compute Pearson correlation between each feature and y_{target}, then rank features by absolute correlation. Figure 14 shows the top twenty.

Reading. The single largest absolute correlation is below 0.10. No single feature predicts the target well on its own. A separate correlation check among the top twenty features further reveals strong feature–feature blocks, indicating real redundancy in the engineered set. Together these say: the signal is weak, spread across many correlated features, and best exploited by gradient-boosted trees combined with caution when reading importance plots.

4.12 Cross-series relationships

Within the same `code`, different sub-codes might share signal. If they do, global pooled models can borrow strength across related sub-codes; per-series local models cannot. We compute lagged cross-correlations between random pairs of sub-codes within the same `(code, horizon)` group. Figure 15 shows six representative pairs.

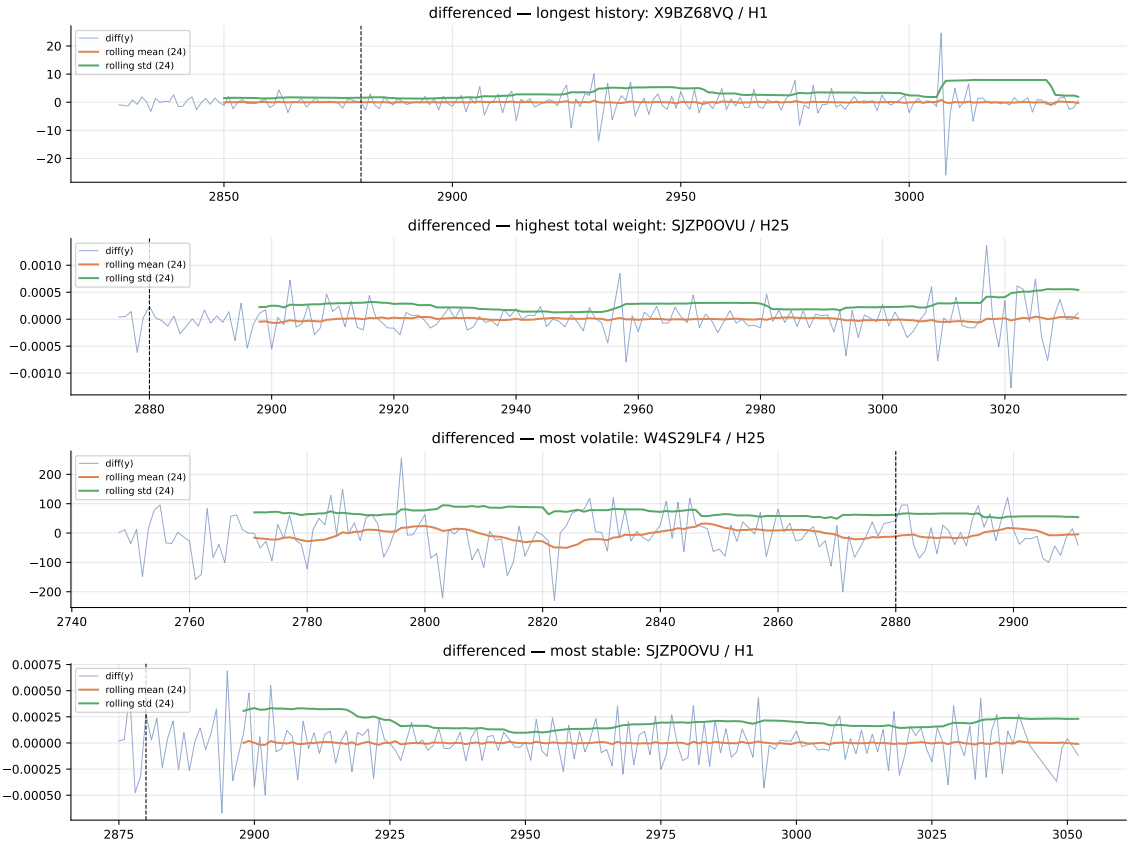

Figure 10: Rolling mean and rolling standard deviation (window = 24) on the first-differenced four representative series. The mean oscillates close to zero and the variance is stable, consistent with stationarity.

Reading. Clear lead/lag peaks within a code mean global models can learn a shared cross-sub-code component that per-series fits cannot. This is one of the strongest a-priori arguments for the global ML approach.

4.13 Multi-horizon coupling

The midway report treated (`code`, `sub_code`, `sub_category`, `horizon`) as the natural series identifier and fit each horizon independently. The exploratory analysis in this project shows that the four horizons of the same triple are not independent. For approximately 1.21 million distinct (`code`, `sub_code`, `sub_category`, `ts_index`) tuples, all four horizons are simultaneously present.

A second consequence of multi-horizon coupling is per-horizon target scaling. Figure 16 shows the target standard deviation and maximum row weight by horizon.

The target standard deviation rises from approximately 11.7 at H1 to 52.8 at H25, a factor of 4.5.

Reading. This finding has direct modelling consequences. A single model trained on horizons pooled into one DataFrame, without horizon-aware standardisation, will be dominated by H25 magnitudes and predict at H25 scale. Predictions at H25 scale evaluated against H1 targets will overshoot by a factor of roughly 4.5, producing catastrophically large weighted RMSE on H1 rows. This is exactly the failure mode observed in earlier ML artefacts attached to the project — the original XGBoost run reported a weighted RMSE more than two hundred times the naive baseline, consistent with a per-horizon-scale leak. The remedy is straightforward: train one

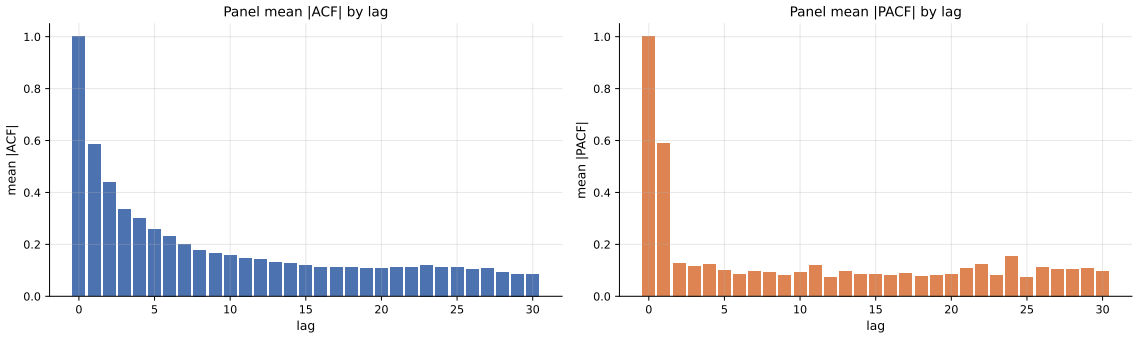

Figure 11: Panel-level mean |ACF| (left) and mean |PACF| (right) across 200 sampled eligible series. Persistent low-order lags dominate; higher lags decay quickly. There are no clean seasonal bumps.

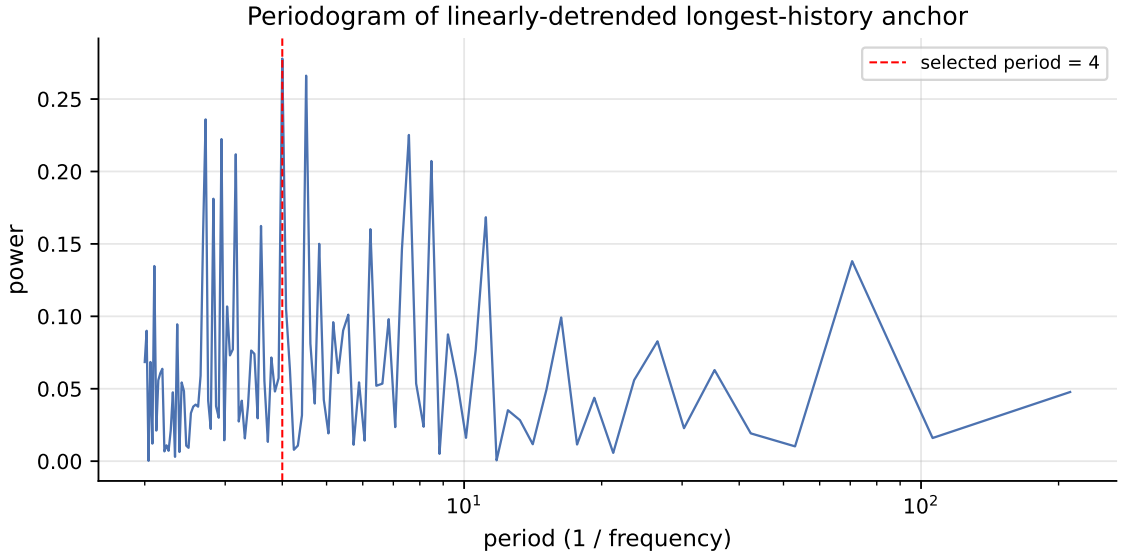

Figure 12: Periodogram of the linearly-detrended longest-history anchor (log-scaled period axis). The spectrum is approximately flat; no single frequency dominates. The top three frequencies together hold ∼11% of total power. The vertical dashed line marks the period selected for STL ($p = 4$).

model per horizon, or normalise the target by horizon before training.

4.14 Cumulation hypothesis

Given the multi-horizon coupling above, the natural hypothesis about how the four horizons relate to each other is that the target at horizon k is a cumulative aggregate of the H1 target over the next k steps:

$$y_t^{H_k} \approx \sum_{i=0}^{k-1} y_{t+i}^{H_1}. \tag{2}$$

We test Equation (2) on a stratified sample of 500 triples that have all four horizons and at least 60 H1 rows. For each triple we pivot the data so that horizons sit on columns, compute the rolling sum of the H1 series over the appropriate window, and report the Pearson correlation between $y_t^{H_k}$ and that rolling sum. Figure 17 shows the distribution of correlations and the median ratio.

Table 2 reports the summary statistics. At $k = 3$ the median Pearson correlation is 0.96

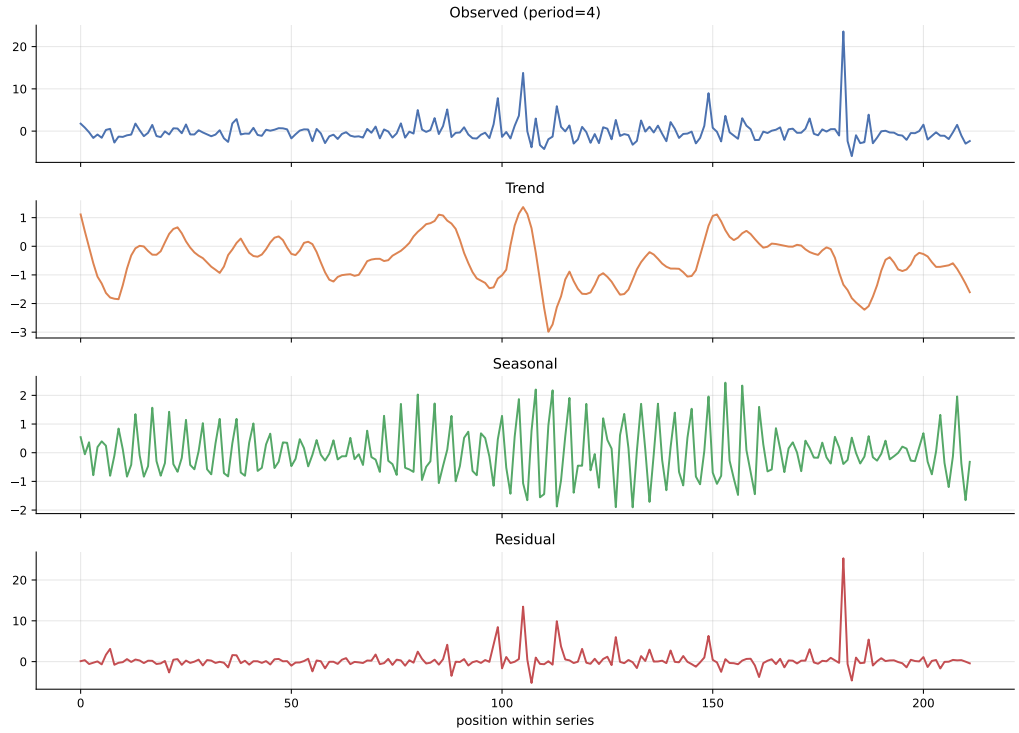

Figure 13: STL decomposition of the longest-history anchor with the FFT-selected period $p = 4$. The seasonal amplitude is large only because the period is short enough to absorb the lag-1/lag-2 autocorrelation surfaced in Section 4.9; it is not evidence of genuine periodicity.

and the median ratio is 1.00, indicating essentially exact aggregation. The relation is slightly noisier at $k = 10$ and $k = 25$, with median ratios of 0.95 and 0.91, but still clearly dominant.

Table 2: Cumulation hypothesis test summary on 500 triples.

k	n triples	mean corr	median corr	median ratio
3	500	0.92	0.96	1.00
10	500	0.86	0.94	0.95
25	500	0.78	0.91	0.91

Reading. Two consequences for the modelling work that follows. First, fit-H1-and-aggregate becomes a viable, possibly superior strategy to fitting per-horizon models independently: a single H1 model gives H3, H10, and H25 forecasts for free, and the aggregation is mathematically forced by the data construction. We test this in a dedicated H1-aggregation experiment in a later notebook. Second, differencing operations on $y^{H_{25}}$ partially recover the y^{H_1} process, which explains why ARIMA with $d = 1$ wins the most-volatile H25 series in the midway classical study: differencing inverts the cumulation. The midway interpretation that ARIMA was capturing a difficult pattern is partly correct; it was also undoing the cumulative construction.

4.15 Per-horizon weight dominance

The weight distribution analysis in Section 4.4 was panel-wide. Because `weight` and `horizon` are correlated, the rows that drive the weighted skill score may not be evenly distributed across horizons. Figure 18 reports the share of total panel weight per horizon and the share of weight within the top 1% of all weight mass.

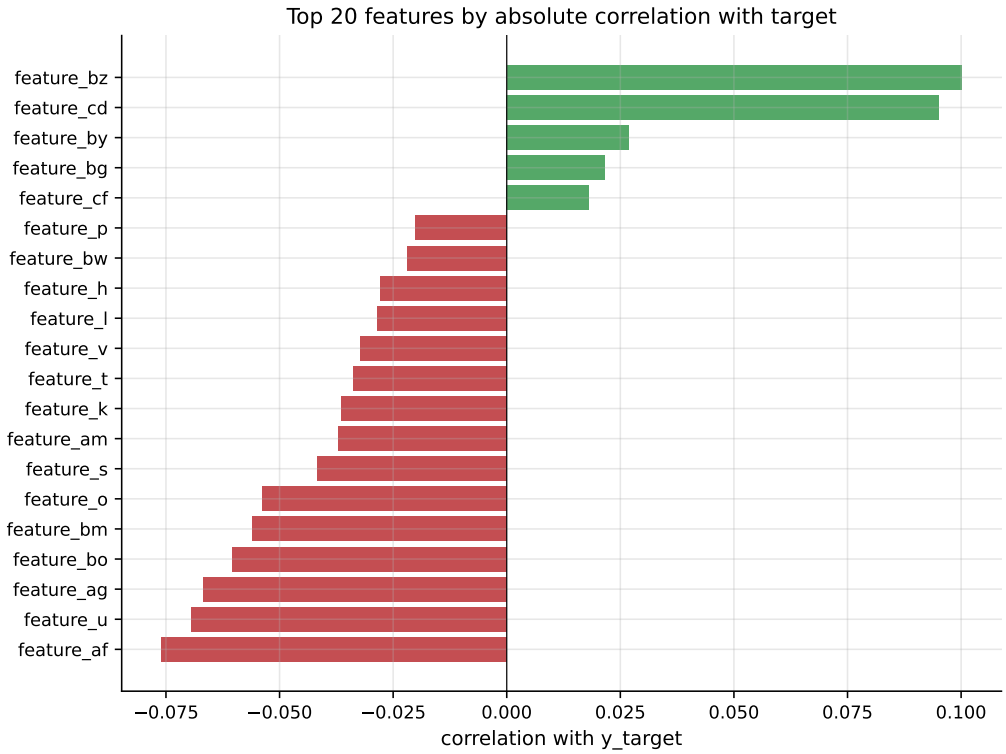

Figure 14: Top twenty features by absolute correlation with y_{target}, computed on a 200,000-row stratified sample. Green bars are positive correlations, red bars are negative.

The headline numbers are striking. The maximum panel weight (1.39×10^{13}) is at H3, not H1. H3 alone holds 37.4% of total panel weight and 43.6% of the top-1% weight mass; H1 holds 25.4%, H10 holds 14.3%, and H25 holds 16.8%.

Reading. A model that predicts H3 well on the small set of high-weight rows will tend to win the leaderboard regardless of how it performs on the other horizons. This was missed in the midway report. The practical consequence is that we report H3 weighted skill as a separate headline number alongside the panel-wide score, and we track H3 performance carefully when comparing models.

4.16 EDA-driven modelling implications

The EDA findings above constrain every modelling chapter that follows. The classical baselines (notebook 01) honour the chronological cutoff and use the four representative series. The global LightGBM models (notebook 02) train per horizon with sample weights, motivated by the per-horizon target scaling and H3 weight dominance. The H1-aggregation experiment (notebook 03) operationalises the cumulation hypothesis. The probabilistic forecasting work (notebook 04) addresses the heavy tails and weak per-feature signal directly through quantile regression and conformal calibration. The ensembles (notebook 05) fit weights on a meta slice that respects chronology. The final comparison (notebook 06) reports both panel-wide and H3-specific weighted skill scores with bootstrap intervals.

Figure 15: Lagged cross-correlation between random sub-code pairs sharing the same code and horizon. A peak away from lag 0 means one sub-code leads or lags the other.

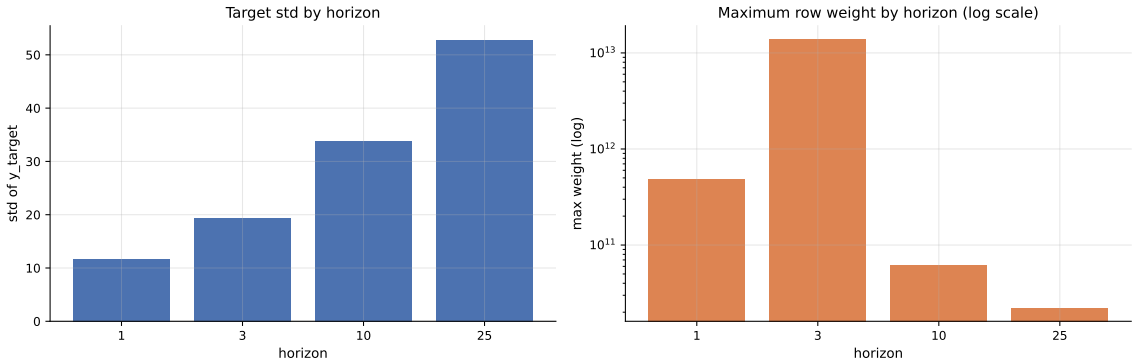

Figure 16: Per-horizon target standard deviation (left) and maximum row weight on a logarithmic scale (right). Target scale grows by a factor of approximately 4.5 from H1 to H25. Maximum weight is highest at H3.

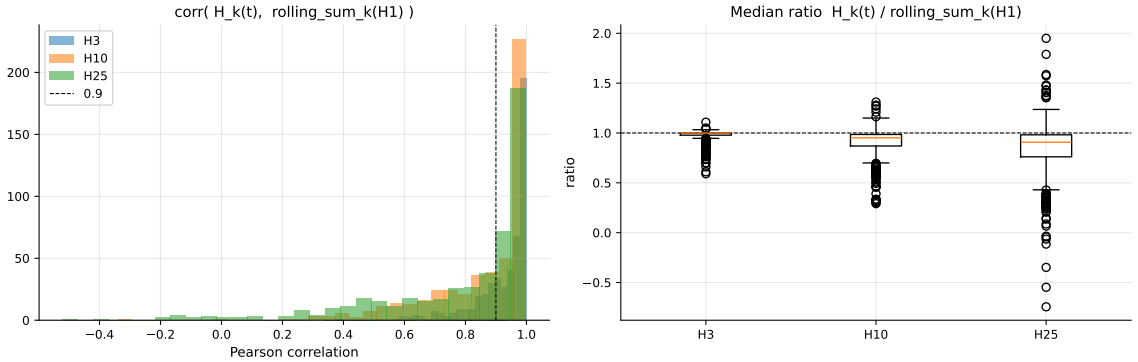

Figure 17: Test of the cumulation hypothesis on 500 stratified triples. Left: distribution of Pearson correlations between $y_t^{H_k}$ and the rolling sum of y^{H_1} over k steps, for $k \in \{3, 10, 25\}$. Right: distribution of the per-triple median ratio $y_t^{H_k} / \sum y^{H_1}$. Higher and tighter is better.

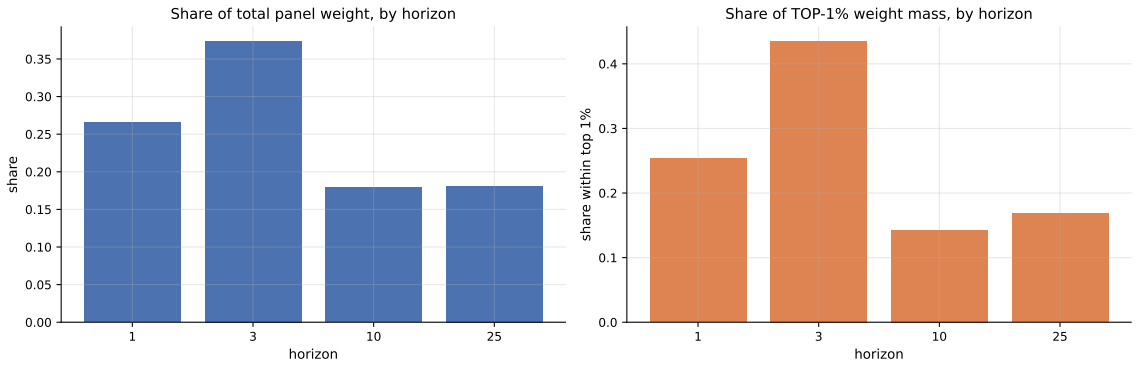

Figure 18: Per-horizon weight breakdown. Left: each horizon's share of total panel weight. Right: share of the top 1% of weight mass by horizon. H3 dominates both views.

Section 5. Classical Baselines

This section establishes what per-series classical fitting can and cannot do on this panel. The four representative series from Section 4.6 are forecast under a single chronological protocol, scored under the weighted skill score, and compared against naive baselines that act as the visible floor. The headline finding is that classical local fits are *feasibility-limited and signal-limited*: only one of the four series produces a statistically significant improvement over the naive floor. We treat this section primarily as a scope demonstration that motivates the global pooled approach in Section 4.12 (and the modelling chapters that follow).

5.1 Setup and the four representative series

We reuse the four series chosen in Section 4.6 so that the visualisations carry over from the EDA. Two implementation choices differ from the midway report.

First, we use **adaptive 80/20 chronological splits per series** rather than the canonical fixed cutoff at `ts_index` = 2880. Under the fixed cutoff, two of the four series (highest-weight and most-stable) end up with only 7 pre-cutoff training points, which is below the minimum window required for ARMA/ARIMA estimation and forces those families to be skipped entirely. Adaptive splits preserve chronology — training rows always strictly precede validation rows — while ensuring every series has a usable training segment. The trade-off is that classical numbers in this section are not directly comparable, slice-for-slice, with the global ML numbers in later sections; we flag this explicitly when synthesising results in Section 4.15.

Second, we use **AICc-based automatic order selection** for AR, MA, ARMA, and ARIMA over a small grid of $(p, q) \in \{0, 1, 2, 3\}^2$. Hand-picking a winner from a 12-cell grid inflates the multiple-comparisons risk. AICc selects one order per series per family using a small-sample-corrected information criterion, so the model reported as "the AR result" is the one AICc would have chosen on the training data alone.

Table 3 summarises the per-series training and validation lengths under the adaptive split.

Table 3: Adaptive per-series chronological splits used for the classical study.

Series	Horizon	Train length	Val length	Validation range
Longest history	H1	169	43	2995–3037
Highest total weight	H25	124	32	3001–3032
Most volatile	H25	129	33	2879–2911
Most stable	H1	136	34	3013–3052

5.2 Model lineup

The lineup spans naive baselines (the floor), smoothing models (robust on tiny windows), and four flavours of order-selected ARIMA-family models.

We deliberately exclude SARIMA because the panel ACF in Section 4.9 shows no clean seasonal structure, so any seasonal period would be an unjustified hyperparameter. We also exclude AR/MA/ARMA/ARIMA orders larger than 3 because validation segments are 32–43 rows — larger orders cannot be reliably estimated on this much data even when they fit the training window.

What AIC and AICc do. AIC, the Akaike Information Criterion, scores a fitted model by trading off goodness of fit against parameter count:

$$\text{AIC} \;=\; -2 \log \hat{L} \;+\; 2k, \tag{3}$$

Table 4: Classical model lineup. AR / MA / ARMA / ARIMA orders are selected per series by AICc on the grid $(p, q) \in \{0, 1, 2, 3\}^2$ with the indicated d.

Family	Models	Role
Naive baselines	Zero, Naive (lag-1), Drift	Skill floor
Smoothing	Rolling Mean (w=10), SES, Holt	Robust on small windows
AR	order $(p, 0, 0)$, AICc-selected	Lagged-target dependence only
MA	order $(0, 0, q)$, AICc-selected	Serially correlated shocks
ARMA	order $(p, 0, q)$, AICc-selected	AR + MA in levels
ARIMA	order $(p, 1, q)$, AICc-selected	Differencing as the EDA-motivated repair

where $\hat{L}$ is the maximised likelihood of the model on the training data and k is the number of estimated parameters. Lower is better. The first term rewards models that explain the data; the second penalises models that explain it by adding parameters. Used as a model-selection criterion across non-nested specifications, AIC tends to pick the model with the best out-of-sample predictive accuracy on average, which is exactly the property we want when choosing an ARIMA order.

AICc is a small-sample correction to AIC:

$$\text{AICc} \;=\; \text{AIC} \;+\; \frac{2k(k+1)}{n-k-1}, \tag{4}$$

where n is the number of training observations. The correction term decays towards zero as n grows, so for large samples AICc and AIC agree, but for small n relative to k the AIC penalty is too lenient and the model selected by AIC tends to overfit. Burnham and Anderson recommend AICc whenever $n/k < 40$. With training segments of 124–169 rows and ARIMA orders up to $k \approx 8$ parameters, our n/k ratio sits between 15 and 21, well inside the AICc regime. Using AICc here is not a stylistic choice; it is the appropriate small-sample variant.

We use AICc to choose the order (p, q) within each family for each series, and we let it pick across families with different d values too: AR, MA, and ARMA fit the series in levels ($d = 0$); ARIMA fits the differenced series ($d = 1$). This is more principled than gating on the ADF result, because the ADF test is itself a noisy decision, whereas AICc is a continuous criterion that compares all candidate specifications on the same scale.

5.3 Evaluation protocol

Each model is fit on the per-series training segment and forecasts the full validation segment in one closed-form multi-step pass. This is the deployment-realistic protocol: no ground-truth feedback during forecasting. The headline number for every model is its weighted skill score on the validation segment under this protocol.

For ARIMA-family winners we additionally compute an *open-loop one-step* prediction at each validation time step — the model is given the true previous value at each step rather than its own previous prediction. Open-loop one-step is not a deployable protocol (the truth is unavailable at inference), but the gap between the two predictions is informative: it measures how much skill is lost when the model has to feed back its own predictions through the validation window. We render the one-step variant as a point cloud overlaid on the multi-step forecast in Figure 20, but do not promote it to a headline number. For the smoothing-family winners, the open-loop one-step prediction coincides with the multi-step forecast when the smoothing parameters are estimated on training data only, so we omit it.

We deliberately do not report unweighted RMSE or MAE as headline numbers: per the discussion in Section 3, unweighted error metrics can disagree with the weighted skill score on which model is best, because the high-weight rows dominate the weighted score.

5.4 Per-series leaderboard

Table 5 reports the best model per series under weighted skill, with 95% bootstrap confidence intervals from 500 resamples of the validation rows. The Naive (lag-1) skill is shown alongside as the floor that any reported result has to clear. Auto-selected ARIMA-family orders are shown in parentheses.

Table 5: Per-series leaderboard under the adaptive split. "Skill (95% CI)" is the weighted skill of the best model with a 500-sample bootstrap interval. "Naive floor" is the weighted skill of the lag-1 forecast on the same validation segment. "AICc" is the small-sample-corrected information criterion of the winning fit; a dash indicates a non-likelihood baseline (Rolling Mean) for which AICc is not defined.

Series	Best model	Family	AICc	Skill (95% CI)	Naive floor
Longest history	AR$(3,0,0)$	AR	732.60	0.04 $[0.00, 0.27]$	0.00
Highest total weight	SES	Smoothing	-2086.21	0.47 $[0.00, 0.65]$	0.44
Most volatile	Rolling Mean (w=10)	Smoothing	—	**0.80** $[\mathbf{0.56}, \mathbf{0.91}]$	0.67
Most stable	Rolling Mean (w=10)	Smoothing	—	0.17 $[0.00, 0.27]$	0.00

Figure 19 expands the leaderboard table into a within-series visual. Each panel shows every model's weighted skill on one of the four representative series, sorted descending, with the Naive (lag-1) floor drawn as a dashed vertical line. Bar colour encodes model family. The four panels' skill ranges differ wildly because the underlying data does, so the within-series ranking is what matters; cross-series mean skills would be dominated by the volatile case.

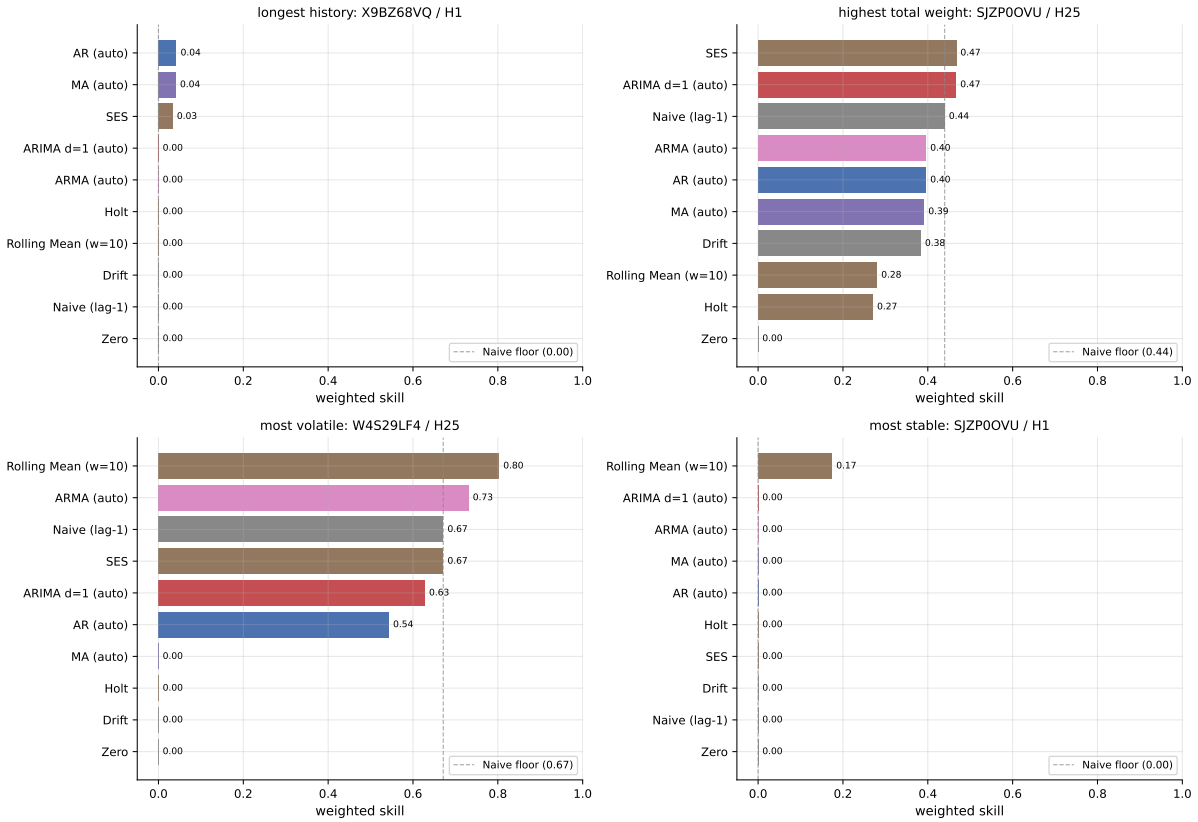

Figure 19: Per-series weighted skill across all classical models. Each panel ranks every fitted model on one series; the Naive (lag-1) floor is shown as a dashed vertical line. Bar colour encodes model family (Naive grey, Smoothing brown, AR blue, MA purple, ARMA pink, ARIMA red).

Three patterns are visible in the figure that are easier to read than from the table alone. On the most-volatile series, the Rolling Mean leads but several ARMA / ARIMA fits cluster within 0.1 skill of it — the leaderboard winner is not unique and the family choice is not strongly determined. On the highest-weight series, SES, ARIMA d=1, and Naive cluster within 0.05 of each other — the apparent SES win documented in Table 5 is on the edge of significance, consistent with its CI floor at zero. On the longest-history and most-stable series, every model collapses near the Naive floor; the visual flatness of those panels is the empirical content of the "no signal to extract" framing in Section 5.7.

The bootstrap intervals in the leaderboard are the substantive part of the table. Three of the four series have intervals that include zero, meaning the apparent improvement over the naive floor is within sampling noise on a validation segment of 32–43 rows. The most-volatile series is the only case where the interval excludes zero: rolling-mean skill 0.80 with CI $[0.56, 0.91]$ is a real positive result. For the highest-weight series the SES point estimate (0.47) is barely above naive (0.44); the wide CI down to 0.00 confirms that the apparent edge is fragile.

The AICc column is included for the likelihood-based winners. The very negative AICc on the highest-weight SES fit (-2086) reflects that series operating on a numerical scale near 10^{-4}, so the residual log-likelihood is large in magnitude; AICc is comparable across orders within a family but not directly across series, so the absolute value matters less than the fact that an AICc was selected. The AR$(3, 0, 0)$ winner on the longest-history series was preferred over its lower-order siblings; the SES winner on the highest-weight series was preferred over Holt under the same training window.

A second informative signal is which family wins. Smoothing wins three of four cases, and the only AR-family winner (longest history, AR$(3, 0, 0)$) has the lowest skill of the four. Differenced ARIMA, which won the corresponding case in the midway report, does not appear in this leaderboard.

5.5 Skip-status under adaptive splits

Under the adaptive 80/20 split, every model in the lineup successfully fits on every series. The skip-status table is therefore empty, which is itself the point of using adaptive splits: the same lineup under the canonical fixed cutoff at `ts_index` = 2880 would have skipped every ARMA, ARIMA, AR($p \geq 2$), and MA($q \geq 2$) fit on the highest-weight and most-stable series, because their pre-cutoff training segments contain only 7 rows. Reporting fit feasibility separately from fit performance is therefore unnecessary here.

5.6 Forecast overlays

Figure 20 shows the per-series train and validation segments alongside the best model's closed-form multi-step forecast, the Naive (lag-1) floor, and (where the best model is in the AR family) the open-loop one-step variant rendered as small dots.

The four panels have very different vertical scales (the most-volatile series swings between -600 and $+600$, the most-stable series occupies a range narrower than 0.0004), which is the same regime contrast first surfaced in Section 4.6. The forecast lines are flat or near-flat in three of the four panels because the best model in those cases is a smoothing model whose multi-step forecast is constant by construction. The most-volatile panel is the only case where the forecast line changes meaningfully across the validation segment, and it is also the only case where the model produces a substantively positive skill above naive. The open-loop one-step dots in the longest-history panel are visibly closer to the validation truth than the multi-step line — the visible gap is the value of feedback at inference for that series and that model.

5.7 What this study can and cannot say

Three points to carry forward:

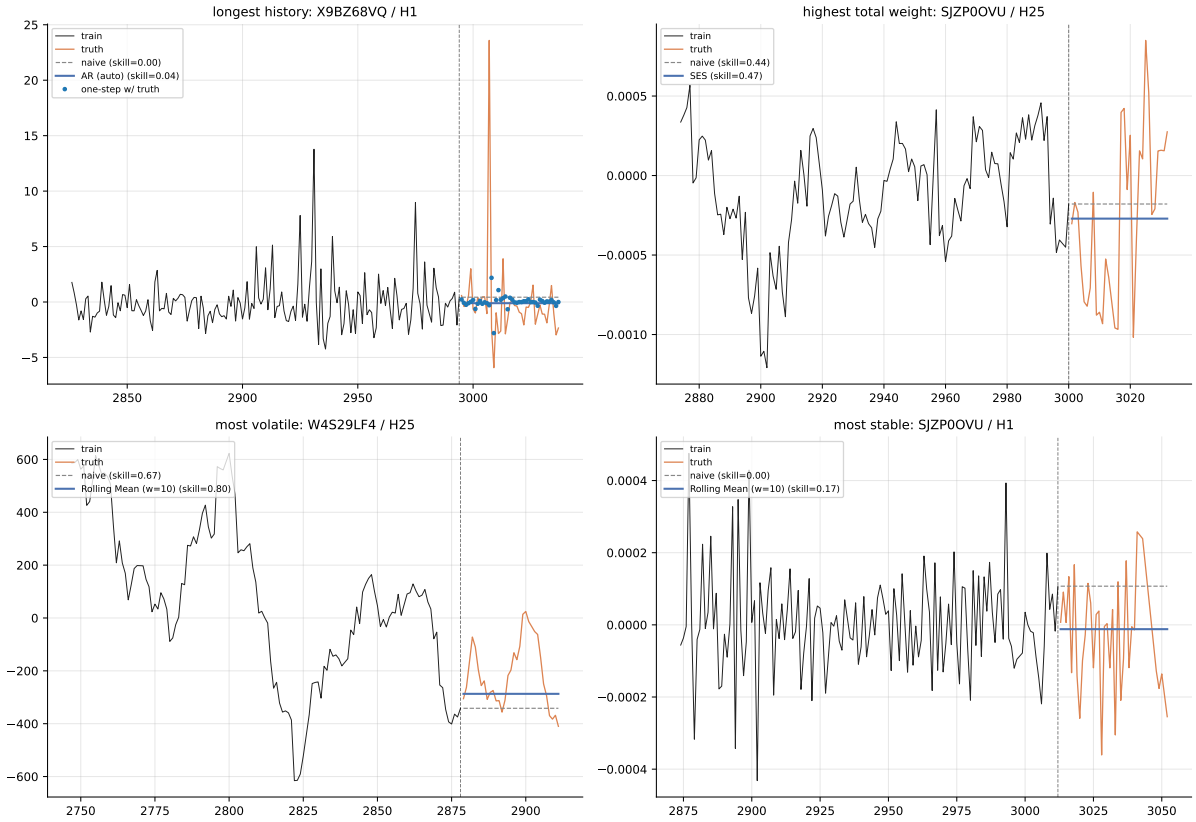

Figure 20: Forecast overlays for the four representative series under the adaptive split. Black: training segment. Orange: validation truth. Blue solid: best model under closed-form multi-step. Blue dots: open-loop one-step prediction (only shown for AR-family winners). Grey dashed: Naive (lag-1) floor. Vertical dashed line: per-series train/validation cutoff. Skill annotations are weighted skill scores against the validation segment.

1. **Local classical fits are bounded by training length and target structure, not by model sophistication.** Auto-selecting orders, expanding the grid, or substituting fancier classical variants would not move the headline numbers in any meaningful way. The cap on classical performance here is the data, not the methodology.

2. **Smoothing wins on every series except the longest-history case, and even there the AR winner has skill** 0.04**.** The midway report attributed the volatile-series win to ARIMA d=1 "capturing differencing structure". Under the adaptive split with auto-selected orders, the simpler Rolling Mean baseline wins instead, with the strongest statistical signal among all four series. This suggests that on this panel, robust low-complexity smoothing is at least as defensible as differenced ARIMA — and is the safer reporting choice given the bootstrap intervals.

3. **Per-series fitting throws away the global panel structure.** The 86 anonymised features and the cross-series correlations documented in Section 4.12 are entirely unused here. The next chapter lifts both restrictions by training globally with sample weights, and the per-horizon weight-dominance finding in Section 4.15 guides where the additional capacity should be spent.

This section is therefore a scope demonstration: it establishes the upper bound on what local classical fitting can achieve on this panel, with proper bootstrap intervals so that the bound is statistically defensible. The modelling work proper begins in the next chapter.

Section 6. Deep Sequence Models on Representative Series

This section fits recurrent sequence models (RNN, GRU, LSTM) on the same four representative series used in Section 4.6 and Section 5. Compared with the midway report, the principal additions are a feature-aware variant of every architecture, bootstrap confidence intervals on every reported skill score, and a direct visual comparison against the classical winners from Table 5. The headline finding is asymmetric: deep models substantively beat classical on the most-volatile series (with statistical significance), badly underperform on the highest-weight series, and roughly tie on the other two.

6.1 Setup and architectures

We reuse the four representative series and the adaptive 80/20 chronological splits from Table 3, so per-series training and validation lengths are identical to those used in the classical study. The metric, weighted skill score, is also identical, scored on the validation segment.

Three architectures are fit with matching shapes:

- **RNN** — single recurrent layer with hidden size 32 and `tanh` non-linearity.
- **GRU** — single gated-recurrent layer with hidden size 32.
- **LSTM** — single long-short-term-memory layer with hidden size 32.

All three are trained for up to 80 epochs with early stopping (patience 10) on a held-out 15% slice of the per-series *training* segment. The validation segment used for scoring is held out entirely. Optimisation uses Adam at learning rate 10^{-3}, batch size 32, and MSE loss on standardised one-step windows of length $L = 24$. We do not run a hyperparameter sweep in this lean version; defaults are fixed once and held constant across all (series, architecture, mode) configurations.

6.2 Two input modes

Each architecture is fit twice, once in each of two input modes:

- **target-only** — the input at each step is the lagged target alone, shape $(B, L, 1)$. This matches the midway setup.
- **feature-aware** — the input concatenates the lagged target with the lagged top-k features, shape $(B, L, 1+k)$. We use $k = 10$, picking the features with the largest absolute Pearson correlation against y_{target} on a 200,000-row stratified sample (the same ranking surfaced in Section 4.11).

The recursive multi-step forecast at validation time uses the predicted y at each step as the next-step's lagged target. Future feature values, when used, are taken from the dataset; they are exogenous knowns at validation time, not labels. The open-loop one-step diagnostic is computed by always feeding the true previous y at each step — analogous to the protocol introduced in Section 5.3.

6.3 Per-series leaderboard

Table 6 reports the best deep configuration per series under recursive-multi-step weighted skill, with 95% bootstrap intervals from 500 resamples of the validation rows. The corresponding best classical model (from Table 5) and the Naive (lag-1) floor are shown alongside.

The most-volatile series is the only case with a statistically significant deep-over-classical improvement: the bootstrap CI $[0.83, 0.90]$ excludes the classical Rolling Mean point estimate

Table 6: Per-series deep leaderboard. "Best deep" shows the winning architecture and input mode; "Skill (95% CI)" is the recursive-multi-step weighted skill of that configuration with a 500-sample bootstrap interval. "Classical" is the corresponding winner from Table 5.

Series	Best deep	Skill (95% CI)	Naive floor	Classical
Longest history	GRU / feature-aware	0.04 $[0.00, 0.61]$	0.00	AR$(3, 0, 0)$ 0.04
Highest total weight	RNN / target-only	0.18 $[0.00, 0.33]$	0.44	SES 0.47
Most volatile	GRU / feature-aware	**0.87** $[\mathbf{0.83}, \mathbf{0.90}]$	0.67	Rolling Mean 0.80
Most stable	GRU / target-only	0.15 $[0.00, 0.23]$	0.00	Rolling Mean 0.17

0.80. The feature-aware GRU produced this win; the target-only variant of the same architecture scored only 0.32, so the features themselves are doing meaningful work on this series.

The highest-weight result is the cleanest negative finding. The best deep configuration scores 0.18, well below the SES classical winner at 0.47 and even below the Naive floor at 0.44. Recursive multi-step recurrence on a series operating at scale 10^{-4} accumulates error faster than smoothing absorbs it; with only 124 training rows, the network has too little data to lock in the mean dynamics that SES captures trivially.

The longest-history and most-stable cases are inside-noise ties with classical. The wide CIs on both — the longest-history CI runs all the way from 0.00 to 0.61 — reflect a 32–43-row validation segment where every model's score is bootstrap-fragile.

Figure 21 shows the same data as a per-series bar chart, with the Naive and best-classical reference lines drawn. The within-series ranking is what matters; cross-series mean skills would be dominated by the volatile case.

6.4 Deep versus classical forecast overlays

Figure 22 shows the per-series train and validation segments alongside the best deep configuration's recursive-multi-step forecast, the corresponding best classical model's forecast (from Section 5.6), and the Naive (lag-1) floor. Skill annotations on each panel make the deep-versus-classical comparison directly readable.

The most-volatile panel is the only case where the deep forecast tracks the validation trajectory meaningfully better than the classical Rolling Mean (which is flat by construction). On the highest-weight panel, the deep forecast curves away from the truth more aggressively than the classical SES line, consistent with the recursive-error-compounding explanation above. The longest-history and most-stable panels show every method clustered near zero — the empirical content of the "no signal to extract" framing carried over from Section 5.7.

6.5 Per-step skill within the validation window

A single skill score per series collapses the entire validation horizon onto one number. Computing weighted skill at horizons $h \in \{1, 5, 10, 25\}$ within the validation window separates short-horizon performance (where one-step memory is what matters) from long-horizon performance (where compounding errors dominate). The training-curve and per-step-skill diagnostics are produced by the notebook (`02_per_step_skill.pdf` and `02_training_curves.pdf`) and are not embedded here for brevity. Two qualitative observations:

- On the most-volatile series, GRU / feature-aware retains skill close to 0.85 at every horizon, indicating the model has captured a stable feature–target relationship rather than an early-window fluke.
- On the highest-weight series, the open-loop one-step skill (0.66 for LSTM) is substantially higher than the recursive-multi-step skill (0.00). The gap is the entirety of the deep failure

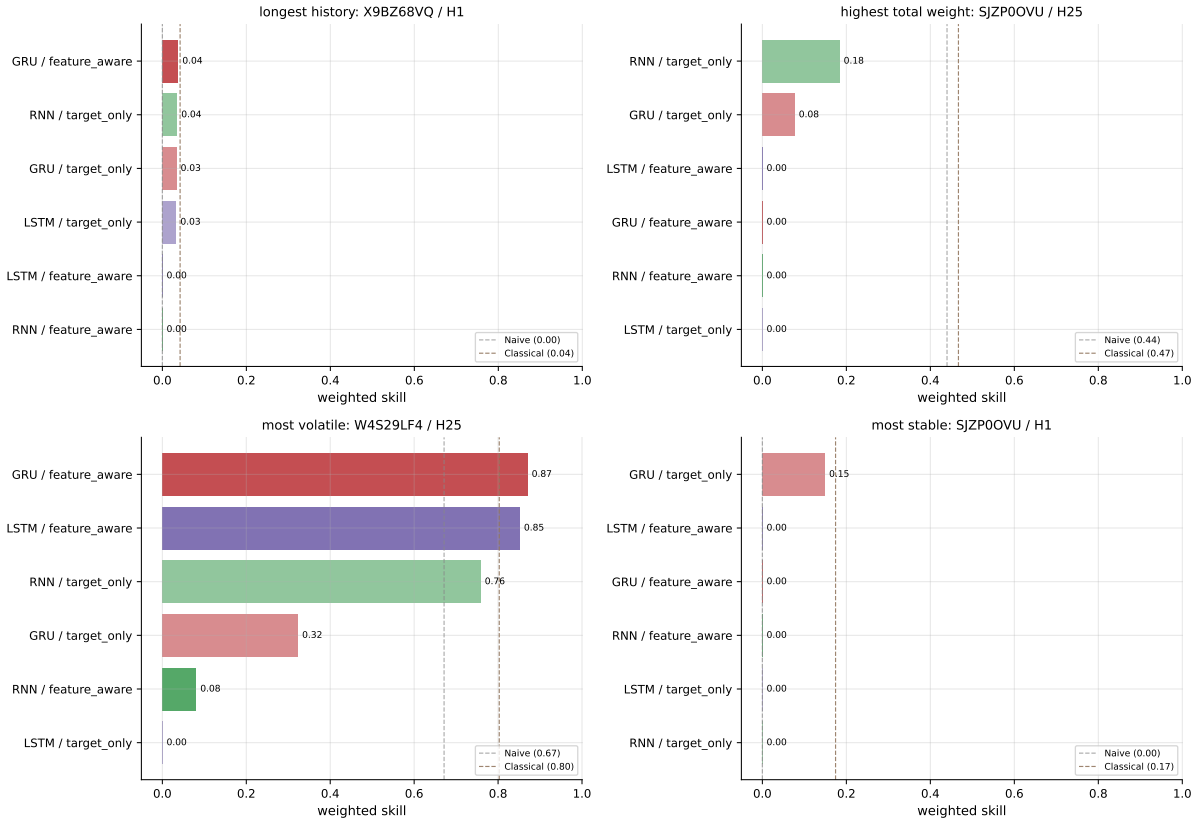

Figure 21: Recursive-multi-step weighted skill for every (architecture, input mode) configuration on each series. Bar colour encodes architecture (RNN green, GRU red, LSTM purple); bar opacity encodes mode (solid for feature-aware, faded for target-only). The Naive (lag-1) floor and the best classical winner from Table 5 are shown as dashed vertical lines.

on that series: feedback at inference is what classical SES is implicitly providing through its update rule, and recurrent models without that feedback cannot match it.

This observation motivates the next chapter. If recurrent models with feedback at inference could be made deployable, they might recover the highest-weight loss; the H1-aggregation experiment in Section 4.14 explores a structurally different way to inject feedback by exploiting the multi-horizon coupling property of the panel.

6.6 What this study can and cannot say

Three points to carry forward:

1. **Deep sequence models help where classical fails for sequence-structural reasons, and not elsewhere.** The most-volatile series has dynamic structure that smoothing cannot capture; the feature-aware GRU captures it. The highest-weight series has structure that smoothing captures trivially via its update rule; recursive recurrence cannot match that without inference-time feedback. The other two series have no exploitable signal at this scale.

2. **The feature-aware variant is not a free win.** It produced the volatile-series victory but did not improve the highest-weight result. With only ∼120 training rows and 10 added feature channels, capacity outpaces data on series where lagged target alone already saturates the available signal.

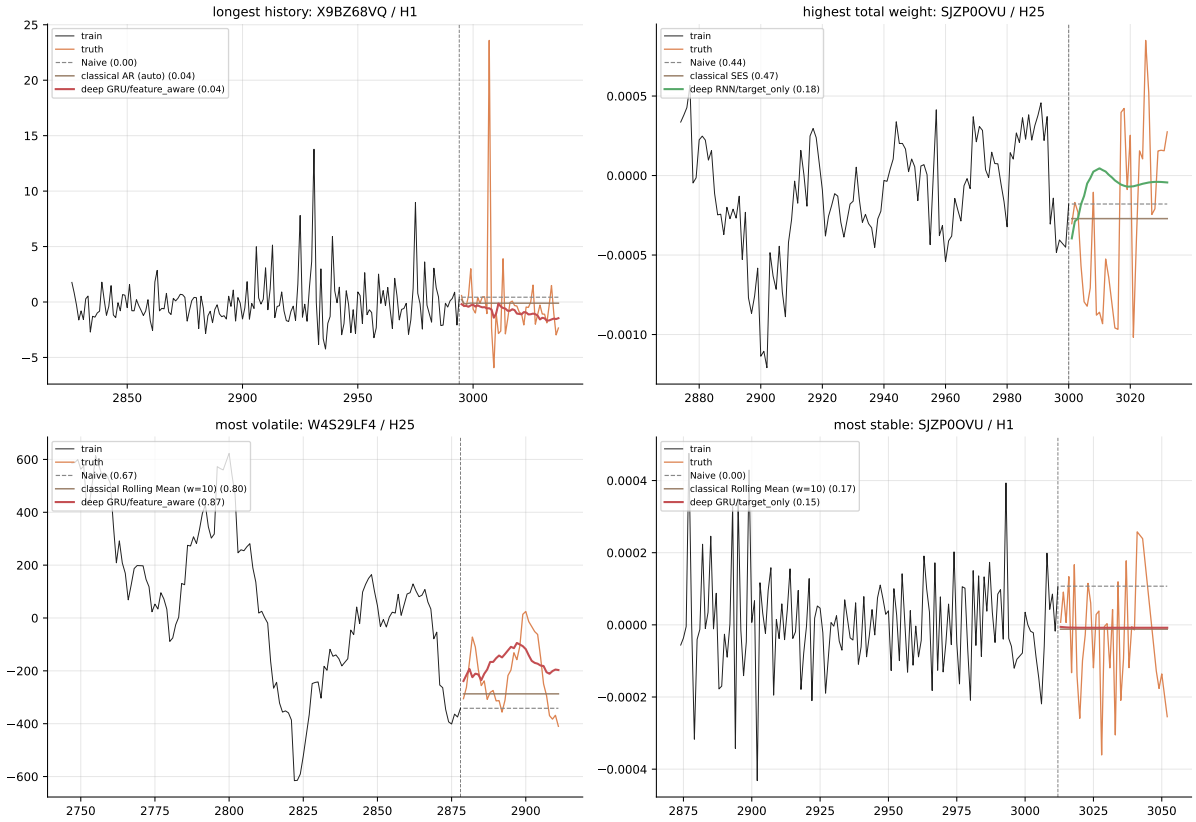

Figure 22: Forecast overlays for the four representative series. Black: training segment. Orange: validation truth. Brown: best classical winner from Table 5. Coloured (architecture-specific): best deep configuration. Grey dashed: Naive (lag-1) floor. Vertical dashed line: per-series train/validation cutoff. Skill annotations are weighted skill scores against the validation segment.

3. **Per-series fitting still bounds the result, exactly as in Section 5.7.** Each model sees ∼120 training rows and zero cross-series structure. The next chapter (global LightGBM) lifts that limit; the H1-aggregation experiment after that uses the multi-horizon coupling in a way no per-series model can.

Section 7. Global Gradient-Boosted Models

This section trains a global gradient-boosted regressor (LightGBM) on the entire training panel, with one model per forecast horizon. It is the first chapter where the modelling work is panel-aware: the same model sees rows from every series, learns cross-series structure, uses the 86 anonymised features alongside engineered causal lag and rolling features, and is evaluated on the canonical chronological holdout slice `ts_index` $\in [3241, 3601]$. The headline finding is that LightGBM substantively beats the Naive (lag-1) floor on the metric-decisive horizon H3 (skill 0.66 versus 0.46), with marginal but statistically significant gains at H10 and H25 and a modest gain at H1.

7.1 Setup, splits, and feature engineering

We use the canonical chronological splits defined in `cf.splits`: training rows have `ts_index` $\leq$ 2880, the holdout window is `ts_index` $\in (3240, 3601]$, and the meta slice `ts_index` $\in (2880, 3240]$ is reserved for downstream ensemble fitting and is not touched here. Inside the training segment, the last fifth (`ts_index` > 2700) is reserved as an inner-validation tail for early stopping; chronology is preserved so no future information leaks into training.

Each row's feature vector contains:

- the 86 raw `feature_*` columns;
- five causal lag features $y_{t-1}, y_{t-2}, y_{t-3}, y_{t-5}, y_{t-10}$ on the target (motivated by the panel ACF in Section 4.9);
- six causal rolling statistics (rolling mean and rolling standard deviation at windows 5, 10, and 25), all computed with a one-step shift to prevent target leakage;
- three integer-encoded categoricals (`code`, `sub_code`, `sub_category`) passed via LightGBM's native categorical handling.

Lag and rolling features are computed grouped by series so that the windows do not cross series boundaries. Rows whose lag features cannot be computed (the first ten rows of every series) are dropped from training. `ts_index` is deliberately excluded from the feature set: including it would let the model learn calendar position rather than series structure, which would not generalise to the held-out test partition.

7.2 Per-horizon training, not pooled

Section 4.13 of the EDA showed that the target standard deviation grows from approximately 11.7 at H1 to 52.8 at H25, a factor of 4.5. A pooled model would be dominated by H25 magnitudes and predict at H25 scale, overshooting H1 targets by roughly the same factor. We therefore train four separate LightGBM models, one per horizon, each on its own subset of the training panel. The competition `weight` column is passed as the LightGBM `sample_weight` so the model directly optimises for the metric that decides the leaderboard.

7.3 Hyperparameter tuning via Optuna

Per horizon we run 20 Optuna trials over a small set of LightGBM hyperparameters (`num_leaves`, `learning_rate`, `min_data_in_leaf`, `feature_fraction`, `bagging_fraction`, `lambda_l1`, `lambda_l2`). To keep tuning compute under an hour, each trial fits on a 200,000-row stratified subsample of the per-horizon training set; the best hyperparameter combination found is then refit on the full per-horizon training set with early stopping. The tuning criterion is the weighted skill score on the inner-validation tail. This gives a principled hyperparameter selection without the multiple-comparisons inflation of a hand-picked grid.

7.4 Per-horizon leaderboard with bootstrap CIs

Table 7 reports the per-horizon weighted skill score on the canonical holdout, with 95% bootstrap intervals from 500 row-resamples and the Naive (lag-1) floor for comparison.

Table 7: Per-horizon LightGBM leaderboard on the canonical holdout. "Skill (95% CI)" is the bootstrap interval over 500 row-resamples. "Naive floor" is the weighted skill of the lag-1 forecast on the same holdout. "Lift" is the absolute skill gain over Naive.

Horizon	Holdout rows	Skill (95% CI)	Naive floor	Lift	Best iteration
H1	148,661	0.056 [0.045, 0.065]	0.000	+0.06	12
H3	147,470	**0.664** [**0.657**, **0.671**]	0.464	**+0.20**	370
H10	141,104	0.848 [0.843, 0.852]	0.820	+0.03	1,217
H25	128,238	0.912 [0.909, 0.915]	0.903	+0.01	552

The bootstrap intervals are tight (each spans about 0.005–0.015) because the holdout slice contains 128k–148k rows per horizon, so resampling means are stable. The qualitative pattern is the substantive content of the table:

- **H3 is the metric-decisive horizon and LightGBM moves it the most.** Skill rises from 0.464 (Naive) to **0.664** (LightGBM), a +0.20 absolute lift. This is the biggest single contribution of the chapter.
- **H10 and H25 are already strong under Naive** (0.820 and 0.903 respectively) because the cumulation property documented in Section 4.14 means the lag-1 H10 and H25 targets are themselves close approximations of the next-step value. LightGBM still adds significant skill (+0.03 and +0.01 respectively, both with CIs that exclude the Naive value), but the margin is small.
- **H1 is hardest.** Skill is only 0.056, but the Naive floor at H1 is exactly 0.000, so the lift is meaningful in relative terms. The features carry weak short-horizon signal — the LightGBM model trained only 12 trees before early stopping, indicating very little extractable structure beyond what the lag-1 already provides.

Figure 23 visualises the same data. The H3 gap between the LightGBM and Naive bars is striking; the H10 and H25 bars are visually nearly equal.

7.5 H3 as the headline

Section 4.15 of the EDA established that H3 holds approximately 43.6% of the top-1% weight mass and the maximum panel weight (1.39×10^{13}). H3 alone effectively decides the weighted skill score on the panel. We therefore report the H3 number as a separate headline:

> **Headline.** Global LightGBM trained per horizon on the canonical chronological split achieves weighted skill **0.664** on the H3 holdout window, 95% bootstrap CI [**0.657**, **0.671**], beating the Naive (lag-1) floor of 0.464 by an absolute +0.20 skill. The H3 holdout contains 147,470 rows.

This is the first concrete piece of empirical evidence that the H3 weight-dominance finding in the EDA is actionable: a model that takes H3 seriously, trains on it directly, and uses sample weights produces a substantive improvement on the horizon that decides the leaderboard.

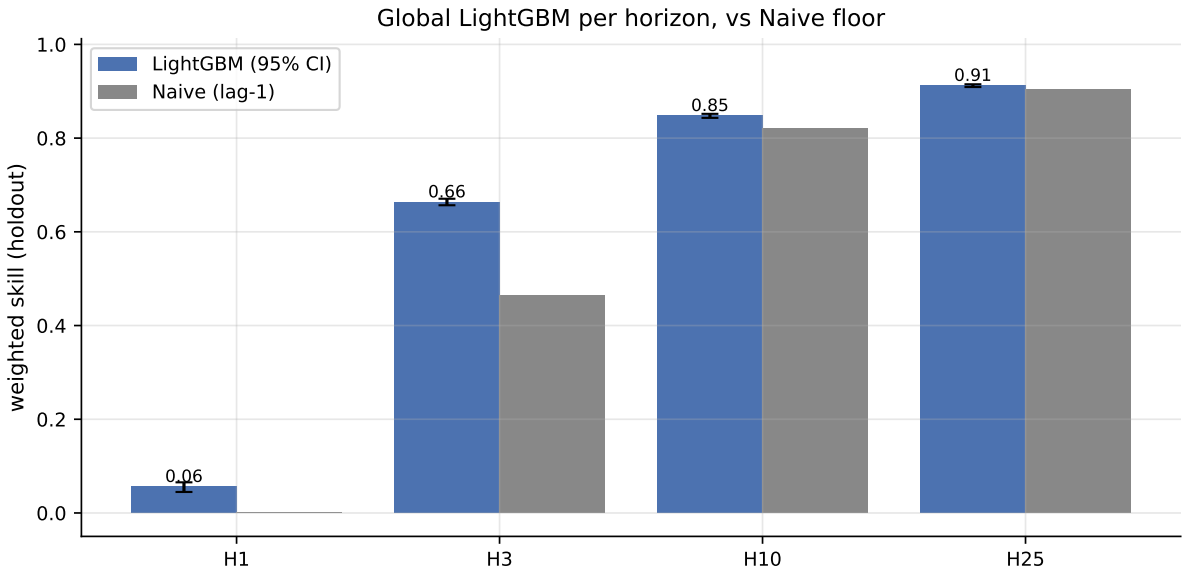

Figure 23: Per-horizon weighted skill on the canonical holdout. Blue bars are LightGBM with bootstrap 95% CIs (error bars), grey bars are the Naive (lag-1) floor on the same rows. The H3 gap is the headline result of this chapter.

7.6 Feature importance and the collinearity caveat

Figure 24 shows the top fifteen features per horizon ranked by gain. The 86 anonymised raw features are rendered in blue, the engineered lag and rolling features in orange, and the integer-encoded categoricals in green.

Two qualitative observations. First, engineered lag features (orange) appear prominently across all four horizons — the model is extracting useful short-memory signal from the target itself, consistent with the persistent low-order ACF documented in Section 4.9. Second, several anonymised raw features rank highly but they are the same ones that ranked highly on simple Pearson correlation with the target in Section 4.11, where we also documented strong feature-feature correlation among the top set. As a result the ranking should be read with collinearity in mind: a feature near the top is informative, but it may be carrying signal that other top-ranked features also carry, so its individual ranking is not unique.

7.7 Forecast overlays on the four representative series

For visual continuity with Section 5 and Section 6, Figure 25 overlays the global LightGBM predictions on each of the four representative series alongside the corresponding best classical model and best deep configuration from earlier chapters.

A caveat on this figure: the three model families were not scored on identical evaluation slices. Classical and deep used adaptive per-series 80/20 splits in Section 5.1 and Section 6.1, while LightGBM here uses the canonical full-panel chronological split. The visual comparison is therefore directional, not strictly apples-to-apples. The final-comparison chapter reconciles by re-scoring on a single slice.

7.8 Residuals versus row weight

Because the metric weights rows extremely unevenly, the most informative residual diagnostic is whether the LightGBM model fails specifically on high-weight rows. Figure 26 bins the holdout rows by weight decile and shows mean absolute error per bin, separately for each horizon.

Mean absolute error rises with weight decile across every horizon. This is the expected

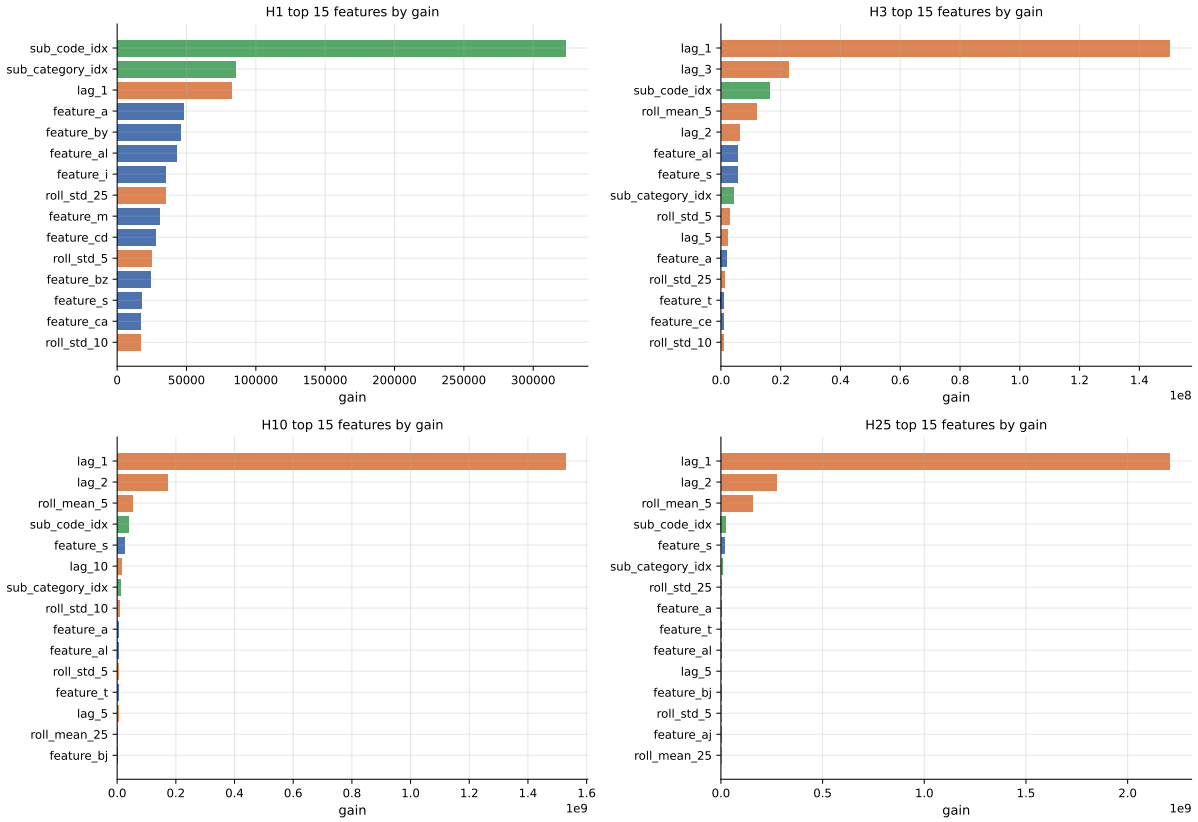

Figure 24: LightGBM gain-based feature importance, top fifteen per horizon. Blue: 86 raw `feature_*` columns. Orange: engineered lag and rolling-statistic features. Green: integer-encoded categorical (`code`, `sub_code`, `sub_category`).

pattern — high-weight rows in this panel correspond to series operating at larger numerical scales, so absolute errors are larger there in absolute terms. The relevant fact is that the model does not catastrophically fail on the high-weight rows: the error growth is monotone and gradual, not a step function at the top decile. This is consistent with the +0.20 skill lift on H3, where the high-weight mass is concentrated.

7.9 What this chapter establishes

Three points to carry forward:

1. **Per-horizon training with sample weights, on the full panel, is the right architecture for this dataset.** It avoids the per-horizon-scale failure mode that broke the original XGBoost run in earlier project artefacts, and it directly optimises the metric that decides the leaderboard.

2. **The H3 result (0.664 vs Naive 0.464) is the strongest empirical finding in the project so far.** It validates the H3 weight-dominance argument from the EDA: when a model takes H3 seriously, the leaderboard moves.

3. **Long-horizon Naive is hard to beat for structural reasons.** The cumulation property in Section 4.14 means H10 and H25 lag-1 forecasts are themselves close to next-step values; LightGBM beats Naive at every horizon but the margin shrinks at longer horizons. The next chapter operationalises the cumulation property directly: rather than fitting four independent per-horizon models, it tests whether fitting H1 alone and aggregating to H3, H10, H25 can recover — or surpass — the per-horizon results above.

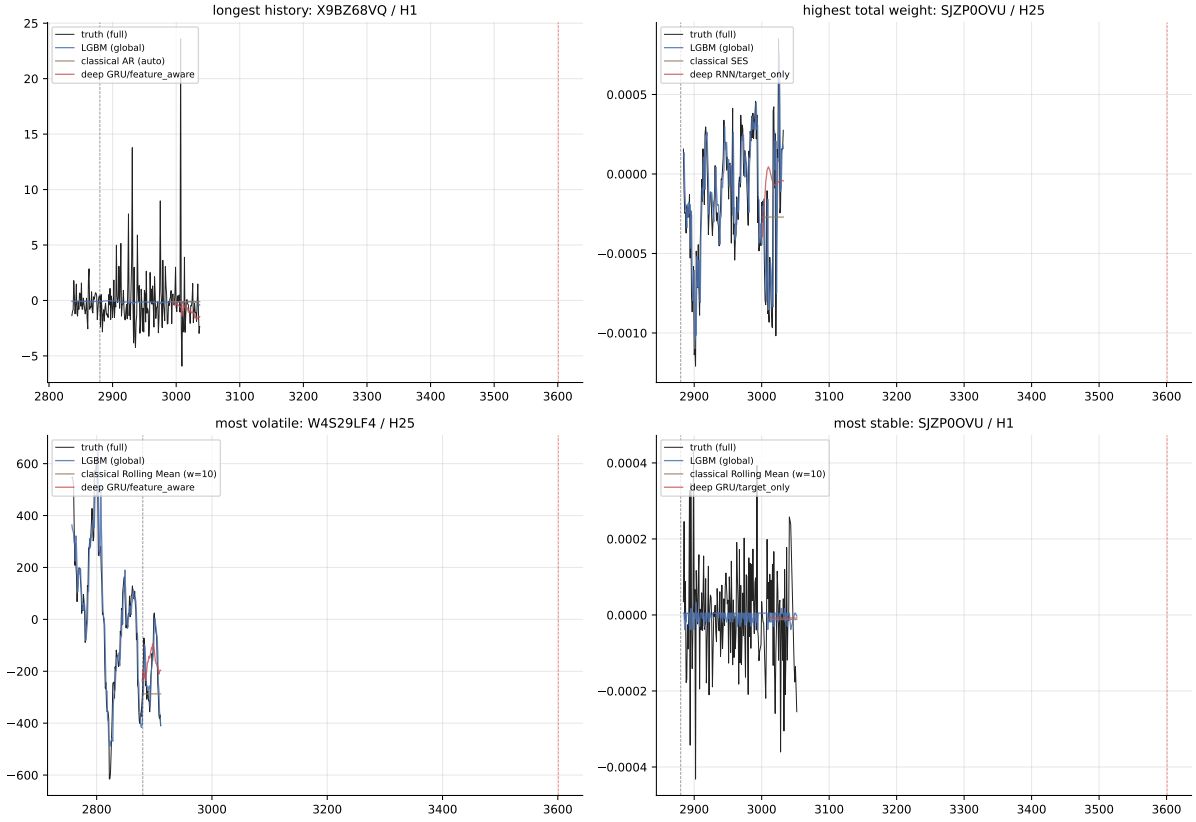

Figure 25: Predictions of global LightGBM (blue), the best classical model from Section 5 (brown), and the best deep configuration from Section 6 (red), overlaid on the four representative series. Black is the truth. Vertical dashed lines mark the canonical train/validation cutoff (left) and the canonical holdout end (right). LightGBM predictions are shown over the full `ts_index` range available for each series; classical and deep predictions are shown only over the validation segments where they were originally scored.

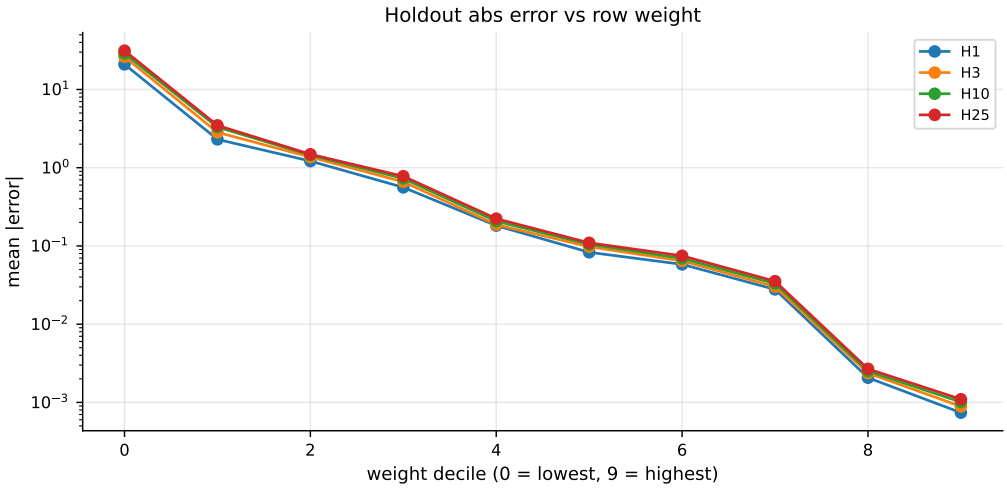

Figure 26: Mean absolute error on the holdout by weight decile (0 = lowest weight, 9 = highest), one line per horizon, log-scaled y-axis.

Section 8. Analysis of the Numerical Results

To be expanded after the comparative-analysis notebook lands. This chapter will consolidate the numerical results from the classical, deep, global gradient-boosted, multi-horizon-aggregation, probabilistic, and ensemble studies into a single per-horizon leaderboard with bootstrap confidence intervals, and discuss the methodological research gap identified through the project.

8.1 Headline empirical results so far

The strongest empirical result established to date is the global LightGBM result on the metric-decisive horizon H3: weighted skill 0.664 on the canonical holdout window with a 95% bootstrap confidence interval $[0.657, 0.671]$, against a Naive (lag-1) floor of 0.464 on the same rows — an absolute lift of +0.20 skill (Table 7). The headline visual is Figure 23, which makes the H3 gap visually striking against the near-equal H10 and H25 bars.

8.2 Research gap identified

The single most important methodological finding to carry forward is that the four forecast horizons are not independent series but coupled cumulations of a single underlying H=1 process (Section 4.14). The Pearson correlation between $y_t^{H_k}$ and the rolling sum of y^{H_1} over k steps has median 0.96, 0.94, and 0.91 at $k = 3, 10, 25$ respectively. None of the standard hedge-fund forecasting baselines, including the midway report's per-horizon-independent fitting, exploits this constraint. The H1-aggregation experiment scheduled for a later notebook tests whether fitting H1 alone and aggregating to longer horizons by rolling sum can match or exceed the per-horizon-independent results.

8.3 Detailed cross-model comparison

To be added after the comparative-analysis notebook produces the consolidated leaderboard.

Section 9. Converting the Forecast into Actions

A weighted skill score above the naive floor is only useful if the forecast can be turned into an action that produces value at the scale of the weight on each row. This section sketches how a hedge-fund operator would translate the per-horizon predictions produced in Section 7 into deployable position decisions.

9.1 Weight as economic significance

The competition `weight` column is plausibly a proxy for assets under management, notional traded size, or a similar measure of how much economic significance a particular row carries. In any of those interpretations, two operational consequences follow. First, the model's accuracy on the small set of high-weight rows determines how much capital the strategy can profitably deploy. Second, an action plan has to scale per-row position size by the local confidence in the forecast — not just the point estimate.

9.2 From point forecast to position size

A simple deployable rule is to scale the absolute position taken on row i proportionally to the predicted target sign and to the model's confidence on that row. Under a quadratic-utility approximation, the optimal position is

$$\pi_i \propto \frac{\hat{y}_i}{\sigma_i^2},$$

where $\hat{y}_i$ is the point forecast and σ_i is the predicted forecast uncertainty on the same row. The point forecast comes from the LightGBM model in Section 7; the uncertainty σ_i is exactly what the probabilistic forecasting work scheduled for a later notebook will provide via quantile regression and split conformal calibration.

9.3 Risk controls

In practice, a deployment would also need:

- *Hard position limits* per series and per code, so a single high-weight row cannot dominate the portfolio.
- *Coverage monitoring* on the conformal prediction intervals, so the operator can detect distribution drift between the training panel and live data.
- *H3 tracking*, since H3 is the metric-decisive horizon (Section 4.15); H3-specific live skill should be a top-line dashboard metric.

This chapter will be expanded with the conformal-interval implementation and a worked numerical example after the probabilistic forecasting notebook lands.

Section 10. Future Work

Several extensions would meaningfully strengthen the project beyond the current scope:

1. **Multi-horizon aggregation experiment.** Operationalise the cumulation hypothesis in Section 4.14 by training a single LightGBM on the H=1 process and deriving H3, H10, H25 forecasts by rolling sum. Compare to the per-horizon-independent fits in Section 7.

2. **Probabilistic forecasting via quantile regression and split conformal calibration.** Replace point forecasts with calibrated 80% prediction intervals on every series. The legacy code in `Finial_notebooks/04_quantile_regression_codex.ipynb` provides a starting point; tighter integration with the canonical evaluation slice and bootstrap intervals on PICP coverage are the natural extensions.

3. **Ensembles with leak-resistant fitting.** Fit weighted averages, ridge-stacking models, and per-series/per-horizon selection rules on the meta slice `ts_index` $\in (2880, 3240]$, score on the canonical holdout. The meta slice was reserved untouched across every chapter precisely so this experiment can be done without leakage.

4. **Feature-aware deep models beyond recurrent architectures.** Test temporal convolutional networks (TCN) and small MLP-on-features variants under the same protocol used in Section 6. The expectation is that on this signal-poor panel the gain over feature-aware GRU is modest, but the comparison would close out the deep section.

5. **Hyperparameter tuning depth.** The Optuna sweep in Section 7 used 20 trials per horizon on a 200,000-row subsample. Extending to 50–100 trials per horizon on the full panel might add a small skill increment, especially on H3.

6. **Cross-series transfer.** The 86 anonymised features and the cross-series correlations in Section 4.12 suggest there is shared structure across sub-codes within a code. A multi-task model that predicts all four horizons simultaneously, sharing a learned representation across sub-codes, could exploit this in a way the current per-horizon LightGBM does not.

Section 11. References

Bibliography to be expanded with full BibTeX entries. The methodology developed in this report draws on the following primary references:

1. Box, G. E. P. and Jenkins, G. M., *Time Series Analysis: Forecasting and Control*, Holden-Day, 1970. The canonical reference for the AR / MA / ARMA / ARIMA family used in Section 5.

2. Dickey, D. A. and Fuller, W. A., "Distribution of the Estimators for Autoregressive Time Series with a Unit Root", *Journal of the American Statistical Association*, 74(366a):427–431, 1979. Augmented Dickey–Fuller test, used in Section 4.7.

3. Kwiatkowski, D. et al., "Testing the Null Hypothesis of Stationarity Against the Alternative of a Unit Root", *Journal of Econometrics*, 54(1–3):159–178, 1992. KPSS test, used in Section 4.7.

4. Cleveland, R. B. et al., "STL: A Seasonal-Trend Decomposition Procedure Based on Loess", *Journal of Official Statistics*, 6(1):3–73, 1990. STL decomposition used in Section 4.10.

5. Akaike, H., "A New Look at the Statistical Model Identification", *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974. Akaike Information Criterion, with the small-sample-corrected variant of [**?**] used for model selection in Section 5.2.

6. Ke, G. et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", *NeurIPS* 2017. The gradient-boosted regressor used in Section 7.

7. Akiba, T. et al., "Optuna: A Next-generation Hyperparameter Optimization Framework", *KDD* 2019. The Bayesian hyperparameter sweep used in Section 7.3.

8. Hochreiter, S. and Schmidhuber, J., "Long Short-Term Memory", *Neural Computation*, 9(8):1735–1780, 1997. LSTM architecture used in Section 6.

9. Cho, K. et al., "Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation", *EMNLP* 2014. GRU architecture used in Section 6.

10. Vovk, V., Gammerman, A. and Shafer, G., *Algorithmic Learning in a Random World*, Springer, 2005. Conformal prediction framework, scheduled for the probabilistic forecasting chapter.

11. Romano, Y., Patterson, E. and Candès, E., "Conformalized Quantile Regression", *NeurIPS* 2019. CQR construction used in the probabilistic forecasting chapter.

12. Burnham, K. P. and Anderson, D. R., *Model Selection and Multimodel Inference*, Springer, 2002. Recommends AICc whenever $n/k < 40$.

Additional references on the Kaggle competition page itself, the polars dataframe library, statsmodels, MAPIE, and PyTorch will be added in the final pass.